

The proofgraph package

Pierre Senellart
pierre@senellart.com

2026/07/29 v1.0.1-dev

Abstract

The `proofgraph` package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, so that most dependencies need no manual annotation; commands are provided for those a visible reference does not capture. The graph is written as a `Graphviz .dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing or reading a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. `proofgraph` builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result. Usually no manual annotation is required, and commands are provided for the dependencies a visible reference does not capture.

The package is best explained by a small example of its use. This manual is itself a `proofgraph` document, so the results stated below are typeset with the package enabled, and the graph shown at the end of this section is the one they produce.

Load `proofgraph` in the preamble, before the `\newtheorem` commands that declare your theorem-like environments (so it sees them as they are created). Here, as in this manual, we pass three options: `autorun` renders the graph with `Graphviz` and embeds it automatically (it needs shell-escape); `cite` captures the `\cites` made inside proofs as dependencies on external work; and `citelabel=tag` labels those citation nodes by their printed tag:

```
\usepackage[autorun,cite,citelabel=tag]{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
\newtheorem{corollary}{Corollary}
```

Then state and prove, as usual, a few results that build on one another:

```
\begin{lemma}\label{lem:even}
  The sum of two even integers is even.
\end{lemma}
```

```

\begin{lemma}\label{lem:odd}
  The sum of two odd integers is even.
\end{lemma}
\begin{theorem}\label{thm:parity}
  The sum of two integers of the same parity is even.
\end{theorem}
\begin{proof}
  By Lemmas~\ref{lem:even} and~\ref{lem:odd}; see~\cite{euclid}.
\end{proof}
\begin{corollary}\label{cor:double}
  For every integer  $n$ , the number  $2n$  is even.
\end{corollary}
\begin{proof}
  Apply Theorem~\ref{thm:parity} to  $n$  and  $n$ .
\end{proof}

```

We obtain the following results, exactly as without the package:

Lemma 1. *The sum of two even integers is even.*

Lemma 2. *The sum of two odd integers is even.*

Theorem 1. *The sum of two integers of the same parity is even.*

Proof. By Lemmas 1 and 2; see [1]. □

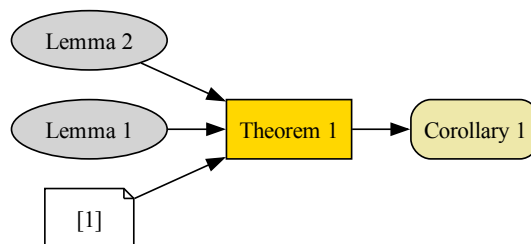
Corollary 1. *For every integer n , the number $2n$ is even.*

Proof. Apply Theorem 1 to n and n . □

In addition, `proofgraph` writes out the dependency graph. With the `autorun` option it is rendered with `Graphviz` and included wherever you call `\proofgraph`:

```
\proofgraph[width=0.7\textwidth]
```

which here produces:



Each cross-reference made inside a proof becomes an arrow *into* the result that proof establishes: the two lemmas point to Theorem 1, which in turn points to Corollary 1. The `\cite` in the proof of Theorem 1 is captured in the same way, thanks to the `cite` option: the external work appears as a distinct node (drawn note-shaped, and labelled here by its printed citation tag, “[1]”, because of `citelabel=tag`) with an arrow into the theorem it helps prove. Moreover, since

this manual loads `hyperref` and `TikZ`, the graph’s nodes are *clickable*: try clicking one to jump to the result it represents (see `\proofgraph` and the `hyperlinks` option).

A handful of commands tune such a graph: `\proofgraphstyle` sets the appearance of each kind of node (the colours here come from it); `\uses` and `\proofgraphedge` record a dependency that no visible `\ref` expresses; `\proofof` attaches a proof to its result; `\proofgraphuntrack` keeps an environment out of the graph; and `\proofgraphignore` and `\proofgraphexclude` drop an unwanted edge or node. The next section puts several of these to work on a real article; Section 3 then documents them, and the package options, in full.

2 A real-world example

The introductory example is deliberately tiny. To show what `proofgraph` produces on a genuine article, the graph in Figure 1 is the one obtained from *Connecting Knowledge Compilation Classes and Width Parameters* [2]. It has 62 numbered results, drawn from five environments (`result`, `theorem`, `proposition`, `corollary` and `lemma`), depends on 24 external works, and contains 114 edges in all.

The graph was produced from the paper essentially as published. Beyond loading the package, the only additions were the following.

- The `cite` option was switched on, and citation nodes labelled by their printed tag, by loading the package as `\usepackage[cite=true,citelabel=tag]{proofgraph}`. The `\cites` made inside proofs then appear as the note-shaped nodes (“[17]”, “[46, Theorem 4.10]”...) along the left of the graph; a work cited for two different results, or for two of its own theorems, gives one node per `\cite` note, so those 23 works appear as 39 nodes.
- One environment was kept out of the graph with a single `\proofgraphuntrack{observation}` (a lone preliminary observation that nothing else uses).
- The five result kinds were styled, with decreasing prominence, by five `\proofgraphstyle` declarations:

```
\proofgraphstyle{result}{shape=box,style="filled,bold",%
                        fillcolor=orange,peripheries=2}
\proofgraphstyle{theorem}{shape=box,style=filled,%
                        fillcolor=gold,penwidth=2}
\proofgraphstyle{corollary}{shape=box,style="rounded,filled",%
                        fillcolor=palegoldenrod}
\proofgraphstyle{proposition}{shape=box,style="rounded,filled",%
                        fillcolor=lightblue}
\proofgraphstyle{lemma}{shape=ellipse,style=filled,%
                        fillcolor=lightgray}
```

- Eleven `\proofgraphedge` commands, recording 27 edges in total, were added for the dependencies that no `\ref` inside a `proof` expresses: eight “obvious” corollaries stated without a `proof` environment (`\proofgraphedge{cor:upper_bound}{thm:upper_bound,...}`), and

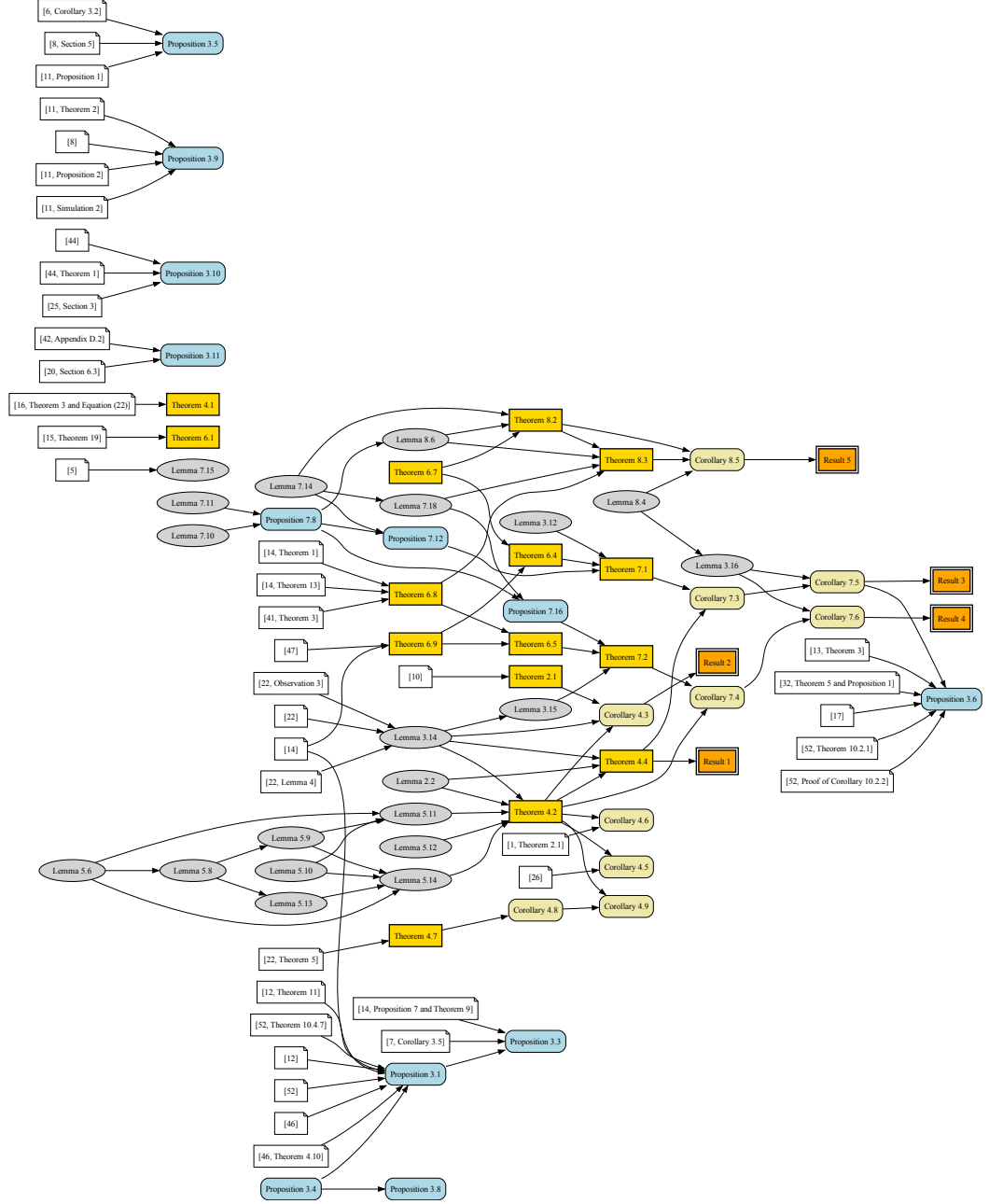


Figure 1: The graph **proofgraph** produces for [2]. The colours and shapes make the structure legible, with the orange double-bordered introduction **results** on the right, fed by the gold theorems, the blue and pale-gold propositions and corollaries, and the grey lemmas.

three theorems whose proof is developed across a whole section rather than inside a single `proof`.

- One `\proofgraphciteedge`, for a lemma stated without a proof, its proof being in the conference version of the article.

Nothing else was annotated: every other edge, and the numbering of all 62 nodes, was inferred automatically from the ordinary cross-references the proofs already contained. In particular, no `\uses`, `\proofof`, `\proofgraphignore` or `\proofgraphexclude` was needed. The five `result` environments that restate the headline results in the introduction (`\begin{result}[(Corollary~\ref{cor:obdd_omega})]...`) are linked to the body results they preview through the `\ref` in their optional title, captured automatically (see the note on reference-carrying titles in Section 3). Two compilation runs (plus `BBTEX`, needed once for the `tag` labels) suffice; the `.dot` file is then rendered with `Graphviz, dot -Tpdf main.dot -o main.pdf`, as with any other document.

3 Usage

3.1 Default behaviour

Load `proofgraph` in the preamble. It only needs to come *after* whatever provides your theorem environments, that is, after `\usepackage{amsthm}` if you load it yourself (some classes, such as `lipics` or `acmart`, provide theorems for you, in which case there is nothing to load first), and *before* the `\newtheorem` commands that declare your theorem-like environments, so that `proofgraph` sees them as they are created:

```
\usepackage{amsthm}      % or nothing, if your class provides theorems
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with `Graphviz`:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

Every *numbered* result of a theorem-like environment you declare becomes a node, and every cross-reference (`\ref`, `\cref`...) inside a proof becomes an edge into the result that proof establishes, with no annotation. The commands captured are `\ref`, `\cref` and `\Cref` (`cleveref`), `\autoref` (`hyperref`), `\eqref` (`amsmath`), `\vref` (`varioref`) and `\zceref` (`zref-clever`, in every form: starred, with options, and with a comma-separated list of labels), each of them whenever it is defined. Purely page-related references (`\pageref`, `\cpageref`, `\zcpageref`) are not captured, as they point at a place rather than at a result; `\uses` records a dependency they would have expressed. Nodes are labelled by the result's formatted name and number ("Theorem 1"), not the environment name. Unnumbered (starred) results are skipped, as they are usually expository or repeated statements; `\proofgraphtrack` draws a chosen one anyway. A cross-reference in the *optional title* of a result is captured the same way: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled $\langle A \rangle$ adds

an edge from $\langle A \rangle$ to `thm:x`, handy for restatements (e.g., an introduction that previews results with `\begin{result}[(Corollary~\ref{cor:y})]`). The *body* of a result is read the same way: a statement that refers to an earlier result, “Under the hypotheses of Theorem 2...”, depends on it just as a proof would. Set `statements=false` to build the graph from proofs alone.

3.2 Recording dependencies by hand

`\uses{<label>}` Inside a proof, records a dependency on the result labelled $\langle label \rangle$ even when no visible reference is made. A comma-separated list is accepted, as in `\uses{lem:a,lem:b}`.

`\proofgraphedge{<A>}{<labels>}` Adds edges recording that result $\langle A \rangle$ depends on the result(s) $\langle labels \rangle$ (a comma-separated list), like a `\uses` but stated outside any proof. Handy for an “obvious” corollary that relies on earlier results without a `proof` environment, as in `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\usescite[<note>]{<keys>}` Inside a proof, the `\uses` of external work: records a dependency on the bibliography entries $\langle keys \rangle$ (a comma-separated list) and draws them as citation nodes, without citing them visibly. The optional $\langle note \rangle$ is the one `\cite` would have taken, as in `\usescite[Thm~3]{knuth}`; it labels the node and, as with a captured `\cite`, tells one result of a work from another.

`\proofgraphciteedge[<note>]{<A>}{<keys>}` The `\proofgraphedge` of citation nodes: records that result $\langle A \rangle$ depends on the cited work(s) $\langle keys \rangle$, stated outside any proof, as in `\proofgraphciteedge{thm:x}{knuth}`. Useful for a result stated without a proof, which has no place to hold a citation. Like `\proofgraphedge` it may appear anywhere, all labels being resolved when the graph is written.

Both commands create the citation node when the work has none yet, and both work whether or not the `cite` option is on: that option only says whether a plain `\cite` is captured on its own. A work reached both ways, once by `\cite` capture and once by hand, still yields a single node.

`\proofof{<label>}` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically: `\begin{proof}` with `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x` (for the `amsthm` proof environment, for `proof` defined as a theorem in the Springer classes, and for deferred proofs as with `apxproof`). When none of this applies, `\proofof` inside the proof pins it to the result labelled $\langle label \rangle$. It pins the proof *as a whole*, wherever in it it stands, so the references made before it count as much as those made after; $\langle label \rangle$ may be the label of a result stated later in the document. This is what a proof written in an appendix, or anywhere else away from its result, needs and all that it needs: its dependencies are then captured as those of a proof placed right after its result would be, and recording them by hand with `\uses` or `\proofgraphedge` is not called for.

3.3 Choosing what appears in the graph

Result environments are detected automatically; these commands adjust which results, and which edges, are kept.

`\proofgraphtrack{⟨names⟩}` Forces the listed environment types to be tracked, overriding the default exclusions, which are `definition`, `example`, `remark` and `note`. Use it in the preamble.

`\proofgraphuntrack{⟨names⟩}` Excludes further environment types, as in `\proofgraphuntrack{observation}`. Use it in the preamble.

`\proofgraphproofenv{⟨names⟩}` Treats the listed environments as proofs, besides `proof`, so that the references they make become dependencies of the result they establish. Use it in the preamble, as in `\proofgraphproofenv{pf}`. It serves a document whose proofs live in an environment of another name, for instance because a proof package had to be renamed to avoid the clash with `amsthm`'s `proof` (as with `pf2`), and equally a `\newtheorem{claimproof}` used for the proofs of claims. Being proofs, the environments named are also removed from the results drawn, even when they are numbered; a `\proofgraphtrack` on the same name overrides that, for an environment to be drawn as a result as well.

An environment taking an optional title is wrapped as `proof` is, so a detached `\begin{pf}[of Theorem~\ref{thm:x}]` attaches to `thm:x`; one declared with `\newtheorem` attaches to the result it follows, or to the one `\proof` of names.

`\proofgraphignore{⟨from⟩}{⟨to⟩}` Suppresses the edge from `⟨from⟩` to `⟨to⟩` (e.g., an unwanted forward citation).

`\proofgraphexclude{⟨label⟩}` Removes the result labelled `⟨label⟩`, together with all its incident edges, from the graph.

`\proofgraphcitecommand{⟨names⟩}` Captures further citation commands, given as a comma-separated list of names without their backslash, as in `\proofgraphcitecommand{citeauthor,citeyear}`. Use it in the preamble. Any command taking the usual `⟨cs⟩* [⟨pre⟩] [⟨post⟩] {⟨keys⟩}` arguments works, including one you define yourself. Naming a command the document does not define is harmless.

It cannot help with the `biblatex multicite` commands (`\cites`, `\parencites`, `\textcites`), whose `(⟨pre⟩)(⟨post⟩)` syntax and repeated key groups `proof-graph` does not parse; cite such works individually, or record them with `\uses`.

3.4 Styling the graph

The attributes are passed verbatim to `Graphviz`; see its attribute reference, <https://graphviz.org/doc/info/attrs.html>, for the node attributes available (and <https://graphviz.org/doc/info/shapes.html> for the shapes).

`\proofgraphstyle{⟨env⟩}{⟨attrs⟩}` Sets `Graphviz` node attributes for all results of the environment `⟨env⟩`, e.g., `shape=ellipse` or `style=filled,fillcolor=gold`.

`\proofgraphstylecite{<attrs>}` Sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), as in `\proofgraphstylecite{shape=box,style=dashed,color=gray}`.

3.5 Embedding the rendered graph

`\proofgraph[<options>]` Includes the rendered graph image at this point, passing the optional `<options>` to the underlying `\includegraphics` (e.g., `\proofgraph[width=\linewidth]`). This is the only feature that needs `graphicx`, which the package therefore does not load on its own: a document that uses `\proofgraph` must load `graphicx` itself (loading `tikz` for the `hyperlinks` overlay also suffices). With the `autorun` option, Graphviz is run at the end of a compilation in which the graph changed (shell-escape required) to produce that image, so `\proofgraph` embeds the graph from the previous run and is up to date after two runs. Without `autorun`, no Graphviz is run: `\proofgraph` embeds a pre-existing `\jobname-proofgraph.<format>` file if you have rendered one under that name, and otherwise warns. When `hyperref` and `TikZ` are both loaded (with `hyperlinks` on and the `-Tplain` file from `autorun` present), every node becomes an internal hyperlink to the result it represents: Graphviz writes the node positions to that `-Tplain` file and `\proofgraph` overlays a transparent `\hyperref` area on each node (needed because the link annotations Graphviz puts in the `.pdf` are discarded by `\includegraphics`).

3.6 Options

`selfloops` Either `remove` (default), dropping edges from a result to itself, or `keep`, retaining them.

`autorun` Boolean (default false). Runs Graphviz automatically at the end of the run; requires compilation with shell-escape enabled. Graphviz is run only when the graph, the engine or the format has changed since the last rendering, so that repeated compilations converge and a build tool such as `latexmk` settles instead of rerunning indefinitely.

`engine` Graphviz layout engine (default `dot`); any Graphviz engine works, such as `neato`, `fdp`, `sfdp`, `circo` or `twopi` (see <https://graphviz.org/docs/layouts/>).

`format` Output format for `autorun` and `\proofgraph` (default `pdf`); any Graphviz output format works, such as `png`, `svg` or `ps` (see <https://graphviz.org/docs/outputs/>).

`hyperlinks` Boolean (default true). Makes the nodes of an embedded graph clickable when `hyperref` and `TikZ` are available (see `\proofgraph`); set it false to embed the graph as a plain image.

`direction` Either `usedby` (default), orienting an edge from the result that is used to the result that uses it (so an arrow `a->b` reads “a is used by b”), or `uses`, the reverse.

`rankdir` Graphviz graph direction (default `LR`).

cite Boolean (default false). Also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes (Section 2), and, unless **statements** is off, in the body of a result too. A `\cite` note is kept: `\cite[Thm~3]{key}` labels the node “Thm 3, ...”. In the two-optional-argument form of `natbib` and `biblatex`, `\cite[see][Thm~3]{key}`, the post-note likewise labels the node and the pre-note is ignored. Every citation command of L^AT_EX, `natbib` and `biblatex` that cites a work is captured the same way, whenever the document defines it, starred variants included: `\citett`, `\citep`, `\citealt`, `\citealp`, `\parencite`, `\textcite`, `\autocite`, `\footcite`, `\footcitett`, `\smartcite`, `\supercite`, `\fullcite` and `\footfullcite`, together with the commands printing only a fragment of a citation (`\citeauthor`, `\citefullauthor`, `\citett`, `\citeyear`, `\citeyearpar`, `\citedate`, `\citeurl`, `\citenum`) and all their capitalised forms. A work invoked twice in one proof, by `\citeauthor` and by `\cite` say, still yields a single edge. `\proofgraphcitecommand` adds any further command; `\usescite` and `\proofgraphciteedge` record a citation node by hand, with or without this option.

statements Boolean (default true). Captures cross-references, and citations when `cite` is on, in the *body* of a result as well, not only in its proof and in its optional title: a statement reading “Under the hypotheses of Theorem 2...” or “... the bound of [?, Thm 3]” records that dependency on the result being stated, as a proof making the same reference would. The capture stops at the `\end` of the result, so references in the surrounding text record nothing. Set it false for a graph built from proofs alone, which is the right reading when statements refer to earlier results to contrast with them rather than to rely on them.

citelabel Either `key` (default), labelling citation nodes by their BibT_EX key, or `tag`, using the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]”...), which is only known after a first run (so it needs two runs). It supports the standard, `hyperref`-wrapped and `natbib` (numeric and author–year) bibliographies. `biblatex` is supported in part: the tag is recovered for its numeric styles ([1]) and its alphabetic ones ([Knu68]), but not for its author–year styles, whose tag is a formatted name list that `biblatex` exposes as no single field. Whenever no clean tag can be recovered the node keeps the key, which is always a correct, if terser, label.

The note of a `\cite` is set where a citation prints it, inside the tag and after it: `\cite[pp.~23--27]{knuth}` gives a node labelled “[Knu68, pp. 23–27]”, carrying the dashes of the document, and “knuth, pp. 23–27” when the label is the key.

file Base name of the output file (default `\jobname`).

3.7 Use with `apxproof`

`apxproof` moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, `proofgraph` relies on the hook `\apxproofhook`, which `apxproof` fires inside each deferred proof. This hook is available in `apxproof v1.4.1` and later; load the two packages in either order and no further setup is needed.

With an *earlier* `apxproof` the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured for the proofs deferred to the appendix*, because `proofgraph` cannot tell which appendix proof belongs to which result (dependencies from any proofs left in the main text are still captured as usual). In that case, add the deferred ones manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade `apxproof`.

4 Limitations

- Environments declared through `\newtheorem` (or `\spnewtheorem`) *after* the package is loaded are tracked automatically. Environments predefined by the document class *before* that (as in `lipics`, `acmart` or the Springer classes) are also detected, provided their name is one of the common result names `proofgraph` probes at `\begin{document}` (`theorem`, `lemma`, `proposition`, `corollary`...); an environment with an unusual name predefined before the package is loaded must be added with `\proofgraphtrack`.
- Results must be labelled for references to them to be resolved.
- Two compilation runs are required, as for any cross-reference.

References

- [1] Euclid. *Elements*, Book IX. c. 300 BCE.
- [2] A. Amarilli, F. Capelli, M. Monet, and P. Senellart. Connecting knowledge compilation classes and width parameters. *Theory of Computing Systems*, **64**(5):861–914, 2020. [doi:10.1007/s00224-019-09930-2](https://doi.org/10.1007/s00224-019-09930-2).

5 Implementation

The format required is that of October 2020, the first to provide `\NewDocumentCommand` in the kernel; `expl3`, which the package also uses, has been preloaded since the February 2020 release.

```

1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2020/10/01]
3 \ProvidesPackage{proofgraph}
4 [2026/07/29 v1.0.1-dev Dependency graph of results]

```

5.1 Dependencies

`amsthm` (loaded for its heading hooks) defines the `proof` environment with `\newenvironment`, which aborts with “Command `\proof` already defined” under a class that already provides one (e.g. the Springer `svjour3` `\proof`). When we are about to load `amsthm` ourselves (it is not already loaded), we save the class’s `proof/\endproof` and free them so `amsthm` loads cleanly, then restore the class’s versions afterwards, so its proof styling is kept (and `proofgraph` still hooks that `proof` environment). If `amsthm` is already loaded, `\RequirePackage` is a no-op and `\proof` is left untouched.

```

5 \ifpackageloaded{amsthm}{}{%
6   \ifundefined{proof}{}{%
7     \let\pgph@class@proof\proof \let\pgph@class@endproof\endproof}%
8   \let\proof\relax \let\endproof\relax}
9 \RequirePackage{amsthm}
10 \ifundefined{pgph@class@proof}{}{%
11   \let\proof\pgph@class@proof \let\endproof\pgph@class@endproof}
12 \RequirePackage{etoolbox}
13 \RequirePackage{xstring}
14 \RequirePackage{kvoptions}
15 \RequirePackage{pdftexcmds}

```

`pdftexcmds` provides `\pdf@filemdfivesum`, with which `autorun` recognises an unchanged graph and skips the `Graphviz` run (see `\pgph@maybrender`). On an engine that offers no file checksum the package still works, only without that optimisation.

`graphicx` is needed only to embed a rendered graph with `\proofgraph`; the core `.dot`-writing machinery does not use it. Rather than burden every document with it, we leave it unloaded and check for `\includegraphics` at the point of use (see `\proofgraph`). When the `hyperlinks` overlay is active `tikz` is loaded, which pulls `graphicx` in anyway.

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite/\cref/\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```

16 \ExplSyntaxOn
17 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
18 \ExplSyntaxOff

```

5.2 Options

```

19 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
20 \DeclareStringOption[remove]{selfloops}

```

```

21 \DeclareBoolOption{autorun}
22 \DeclareStringOption[dot]{engine}
23 \DeclareStringOption[pdf]{format}
24 \DeclareStringOption[LR]{rankdir}
25 \DeclareStringOption[usedby]{direction}
26 \DeclareBoolOption{cite}
27 \DeclareStringOption[key]{citelabel}
28 \DeclareBoolOption[true]{statements}
29 \DeclareBoolOption[true]{hyperlinks}
30 \DeclareStringOption{file}
31 \ProcessKeyvalOptions*
32 \ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi

```

5.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph `{ }` block.

```

33 \begingroup
34   \catcode'\ [=1 \catcode'\ ]=2
35   \catcode'\ {=12 \catcode'\ }=12
36   \gdef\pgph@ob[{}]%
37   \gdef\pgph@cb[{}]%
38 \endgroup

```

5.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

39 \newcount\pgph@idcnt
40 \newcount\pgph@proofcnt
41 \newcount\pgph@citeslotcnt
42 \def\pgph@nodes{}
43 \def\pgph@edges{}
44 \def\pgph@ignores{}
45 \def\pgph@excludes{}
46 \let\pgph@curresult\@empty
47 \let\pgph@curproof\@empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of `hyperref`.

```

48 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}%
49   \ifcsname pgph@rmap@#2\endcsname\else
50     \global\@namedef{pgph@rmap@#2}{#1}\fi}
51 \newcommand\pgph@setmap[2]{%
52   \edef\pgph@tmpid{#2}%
53   \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
54   \ifcsname pgph@rmap@\pgph@tmpid\endcsname\else
55     \global\expandafter\def\csname pgph@rmap@\pgph@tmpid\endcsname{#1}%
56   \fi
57   \protected@write\@auxout{}{\string\pgph@auxmap{#1}{\pgph@tmpid}}%
58 }

```

5.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

59 \let\pgph@candidates\@empty
60 \newcommand\pgph@register[1]{%
61   \edef\pgph@thisname{#1}%
62   \IfBeginWith\pgph@thisname{axp@}{-}{%
63     \ifcsname pgph@cand@\pgph@thisname\endcsname\else
64       \global\@namedef{pgph@cand@\pgph@thisname}{-}%
65       \ifx\pgph@candidates\@empty
66         \global\let\pgph@candidates\pgph@thisname
67       \else
68         \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
69       \fi
70     \fi
71   }%
72 }
```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g., the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

73 \let\pgph@orig@newtheorem\newtheorem
74 \renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
75 \newcommand\pgph@nt@plain[1]{%
76   \pgph@register{#1}\pgph@orig@newtheorem{#1}}
77 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
78 \ifdefined\spnewtheorem
79   \let\pgph@orig@spnewtheorem\spnewtheorem
80   \renewcommand\spnewtheorem{%
81     \@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
82   \newcommand\pgph@spnt@plain[1]{%
83     \pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
84   \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
85 \fi
```

`\proofgraphtrack`
`\proofgraphuntrack`
`\proofgraphtrack`
`\proofgraphuntrack`

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{⟨names⟩}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{⟨names⟩}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the

preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaultexclude` seeds the exclusion set.

```

86 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,%
87   definition,conjecture,claim,fact,remark,example,observation,%
88   notation,problem,question,property,assumption,hypothesis,principle}
89 \newcommand\pgph@defaultexclude{definition,example,remark,note}
90 \newcommand\proofgraphtrack[1]{%
91   \@for\pgph@one:=#1\do{%
92     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
93     \csundef{pgph@xtype@\pgph@one}%
94     \global\@namedef{pgph@force@\pgph@one}{}%
95     \expandafter\pgph@register\expandafter{\pgph@one}%
96   }%
97 }
98 \newcommand\proofgraphuntrack[1]{%
99   \@for\pgph@one:=#1\do{%
100     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
101     \csundef{pgph@force@\pgph@one}%
102     \global\@namedef{pgph@xtype@\pgph@one}{}%
103   }%
104 }
105 \expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

106 \newcommand\pgph@dohook[1]{%
107   \ifcsname pgph@hooked@#1\endcsname\else
108     \global\@namedef{pgph@hooked@#1}{}%
109     \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
110   \fi
111 }
112 \newcommand\pgph@maybehook[1]{%
113   \ifcsname pgph@force@#1\endcsname
114     \pgph@dohook{#1}%
115   \else
116     \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
117   \fi
118 }
119 \newcommand\pgph@resolveregistration[1]{%
120   \@for\pgph@one:=#1\do{%
121     \ifcsname\pgph@one\endcsname%
122       \expandafter\pgph@register\expandafter{\pgph@one}\fi%
123   \ifx\pgph@candidates\@empty\else
124     \@for\pgph@one:=\pgph@candidates\do{%
125       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
126   \fi
127 }
128 \AtBeginDocument{%
129   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames`
`\pgph@recordresult`

the environment name (third argument, also a fallback label), but displayed by the result's *formatted* name ("Theorem", "Lemma"), captured from the heading. We get that name by locally wrapping the heading macros, whose first two arguments are the formatted name and the number. Which macro carries the *optional note* (where a restatement's cross-reference lives) depends on the setup: `amsthm`, which `proofgraph` always loads, routes every heading through `\@begintheorem`, defined as `#1#2[#3]` with the note as the bracketed `#3` (and leaves `\@opargbegintheorem` `\relax`); the L^AT_EX kernel instead uses a two-argument `\@begintheorem` with a separate three-argument `\@opargbegintheorem` for the note; the Springer classes use `\@spbegintheorem`/`\@spopargbegintheorem`. We detect the `amsthm` form (`\@opargbegintheorem` is `\relax`) and read the bracketed note there, otherwise we hook `\@opargbegintheorem`; either way the note goes to `\pgph@scanrefs`. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

The note read as a *delimited* argument is handed back to the original macro inside braces, `[{\langle note \rangle}]` rather than `[⟨note⟩]`. TeX strips the outer braces of a delimited argument that is one brace group in its entirety, so a title written `[{\cite[p.~37]{a-1}}]` – the bracing the `amsthm` manual prescribes to hide the `]` of the citation note – would otherwise reach the original macro unbraced, whose own scan for `]` would then stop inside the citation and take the rest of the note for text. Rebracing restores exactly what was written: the added braces are stripped again by that scan.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

Unless `statements` is switched off, the *body* of the result is treated like the body of a proof of that result (`\pgph@beginstatement`), so that a cross-reference or a citation made in the statement itself becomes a dependency. Those assignments are local to the environment group, as the ones above are, so they revert at `\end`.

```

130 \newcommand\pgph@prependnode[1]{%
131   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
132 }
133 \newcommand\pgph@beginresult[1]{%
134   \global\advance\pgph@idcnt\@ne
135   \xdef\pgph@curresult{\the\pgph@idcnt}%
136   \protected@edef\pgph@tmp{%
137     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
138   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
139   \pgph@hooknames
140   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
141   \let\label\pgph@maplabel
142   \ifpgph@statements\pgph@beginstatement\fi
143 }
144 \newcommand\pgph@hooknames{%
145   \ifdefined\@begintheorem
146     \let\pgph@orig@bt\@begintheorem
147     \ifx\@opargbegintheorem\relax
148       \def\@begintheorem##1##2[#3]{%

```

```

149     \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
150     \pgph@orig@bt{##1}{##2}[-{##3}]]}%
151   \else
152     \def\@begintheorem##1##2{%
153       \pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
154   \fi
155 \fi
156 \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
157   \let\pgph@orig@obt\@opargbegintheorem
158   \def\@opargbegintheorem##1##2##3{%
159     \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
160     \pgph@orig@obt{##1}{##2}{##3}}%
161 \fi\fi
162 \ifdefined\@spbegintheorem
163   \let\pgph@orig@spbt\@spbegintheorem
164   \def\@spbegintheorem##1##2##3##4{%
165     \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
166 \fi
167 \ifdefined\@spopargbegintheorem
168   \let\pgph@orig@spobt\@spopargbegintheorem
169   \def\@spopargbegintheorem##1##2##3##4##5{%
170     \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
171     \pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
172 \fi
173 }

```

`\pgph@scanrefs` A result whose optional title carries a cross-reference, e.g., a `result` whose title is `[(Corollary~\ref{cor:x})]`, is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the capturing commands installed and `\pgph@curproof` pointing at the current result. Everything happens inside a group and `\pgph@curproof` is only *read* while the box is built, so a local assignment suffices and nothing outside the title is affected; a global one would escape the enclosing environment, which under `statements` (where `\pgph@curproof` is already set on entering the result) would leave every subsequent reference attributed to that result.

```

174 \newcommand\pgph@scanrefs[1]{%
175   \ifx\pgph@curresult\@empty\else
176     \begingroup
177       \let\pgph@curproof\pgph@curresult
178       \pgph@installcapture
179       \setbox\z@\hbox{#1}%
180     \endgroup
181   \fi
182 }

```

The formatted name is the first argument of every heading macro (`\@begintheorem`/`\@opargbegintheorem` for `amsthm`/`\newtheorem`, `\@spbegintheorem`/`\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from

`\@currentlabel` rather than from that argument, which some classes pollute (e.g., `lipics` injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

183 \newcommand\pgph@recordresult[2]{%
184   \ifcsname pgph@name@\pgph@curresult\endcsname\else
185     \begingroup
186       \let\protect\@empty
187       \def\axp@forward@link##1##2{##2}%
188       \edef\pgph@tmpname{#1}%
189       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
190         \pgph@tmpname
191       \global\@namedef{pgph@seen@\pgph@curresult}{}%
192       \def\pgph@tmpn{#2}%
193       \ifx\pgph@tmpn\@empty\else
194         \edef\pgph@tmpnum{\@currentlabel}%
195         \global\expandafter\let\csname pgph@num@\pgph@curresult%
196           \endcsname\pgph@tmpnum
197       \fi
198     \endgroup
199   \fi
200 }
```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{<target>}{<number>}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```

201 \newcommand\pgph@maplabel[1]{%
202   \pgph@setmap{#1}{\pgph@curresult}%
203   \ifcsname pgph@num@\pgph@curresult\endcsname\else
204     \ifcsname pgph@seen@\pgph@curresult\endcsname\else
205       \begingroup
206         \let\protect\@empty
207         \def\axp@forward@link##1##2{##2}%
208         \edef\pgph@tmpnum{\@currentlabel}%
209         \global\expandafter\let\csname pgph@num@\pgph@curresult%
210           \endcsname\pgph@tmpnum
211       \endgroup
212     \fi
213   \fi
```

```

214 \pgph@saveditlabel{#1}%
215 }

```

5.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```

216 \newcommand\pgph@adddedge[2]{%
217   \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
218   \expandafter\g@addto@macro\expandafter\pgph@edges%
219     \expandafter{\pgph@tmpedge}%
220 }
221 \newcommand\pgph@recordref[1]{%
222   \ifx\pgph@curproof@empty\else
223     \@for\pgph@one:=#1\do{%
224       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
225       \pgph@adddedge{\pgph@curproof}{\pgph@k}%
226     }
227 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The proof is not recorded as the result it proves but as a *proof instance* of its own, $p\langle n \rangle$, whose owning result is held in `\pgph@owner@p<n>` and read only at emission. An edge captured in the proof therefore does not fix its source when it is seen, which is what lets `\proofof` pin the whole proof and not merely the references that follow it. Were the source fixed at capture, a proof opening with “By Theorem 1” and naming its own result only at the end would give that first reference to whichever result happens to precede the proof. The indirection also lets a proof be pinned to a result labelled later in the document, the label being resolved with all the others when the graph is written.

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g., a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

228 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@savedit@ref{#1}}
229 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@savedit@cref{#1}}
230 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@savedit@Cref{#1}}
231 \protected\def\pgph@cap@autoref#1{%
232   \pgph@recordref{#1}\pgph@savedit@autoref{#1}}
233 \protected\def\pgph@cap@eqref#1{%
234   \pgph@recordref{#1}\pgph@savedit@eqref{#1}}
235 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@savedit@vref{#1}}

```

`zref-clever`’s `\zcref` is the one captured command that is not simply $\langle cs \rangle \{ \langle labels \rangle \}$: it is `\zcref*[\langle options \rangle] \{ \langle labels \rangle \}`, and both the star and the options have to be handed on unchanged. Its mandatory argument is a comma-separated list, which `\pgph@recordref` already splits.

```

236 \NewDocumentCommand\pgph@cap@zcref{s0{}m}{%
237   \pgph@recordref{#3}%
238   \IfBooleanTF{#1}{\pgph@savedit@zcref*{#2}{#3}}{\pgph@savedit@zcref{#2}{#3}}%
239 }
240 \newcommand\pgph@install[1]{%

```

```

241 \ifcsname #1\endcsname
242   \expandafter\ifx\csname #1%
243     \expandafter\endcsname\csname pgph@cap@#1\endcsname
244   \else
245     \expandafter\let\csname pgph@saved@#1%
246       \expandafter\endcsname\csname #1\endcsname
247     \expandafter\let\csname #1%
248       \expandafter\endcsname\csname pgph@cap@#1\endcsname
249   \fi
250 \fi
251 }

```

The set of captured commands is needed in three places (a proof, the optional title of a result, and an `apxproof` deferred proof), so it is named once here: adding a command to the list must not be a matter of remembering three call sites.

```

252 \newcommand\pgph@installrefs{%
253   \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
254   \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
255   \pgph@install{zceref}%
256 }

```

The cross-reference commands and, when the `cite` option is on, the citation commands are always installed together, so the pair has a name of its own: every place that opens a capturing context (a proof, the optional title of a result, and, with `statements`, the body of a result) installs exactly the same set.

```

257 \newcommand\pgph@installcapture{%
258   \pgph@installrefs
259   \ifpgph@cite\pgph@installcite\fi
260 }
261 \newcommand\pgph@beginproof{%
262   \global\advance\pgph@proofcnt\@ne
263   \edef\pgph@curproof{p\the\pgph@proofcnt}%
264   \global\expandafter\let\csname pgph@powner@\pgph@curproof\endcsname
265     \pgph@curresult
266   \pgph@installcapture
267   \pgph@hookproofhead
268 }

```

`\pgph@beginstatement` Under the `statements` option the body of a result captures like the body of its proof. The proof-heading hook is deliberately *not* installed here: it turns a cross-reference in an optional title into a `\proofof`, which is right for a proof titled “of Theorem 1” but not for a result titled “(restates Theorem 1)”, where the reference is an ordinary dependency.

```

269 \newcommand\pgph@beginstatement{%
270   \let\pgph@curproof\pgph@curresult
271   \pgph@installcapture
272 }

```

When the `proof` environment is itself a theorem-like environment (as with Springer’s `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as [of Theorem~\ref{thm:x}] sets `\proofof{thm:x}`, attributing the proof to its result. As in `\pgph@hooknames`, a `\relax \@opargbegintheorem` is left alone: that is the `amsthm` setup, where

the macro is never called, and giving it a meaning would make `\pgph@hooknames` read a result nested in the proof as having the kernel's two-argument heading rather than `amsthm`'s.

```

273 \newcommand\pgph@hookproofhead{%
274   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
275     \let\pgph@orig@obtp\@opargbegintheorem
276     \def\@opargbegintheorem##1##2##3{%
277       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
278   \fi\fi
279   \ifdefined\@spopargbegintheorem
280     \let\pgph@orig@spobtp\@spopargbegintheorem
281     \def\@spopargbegintheorem##1##2##3##4##5{%
282       \pgph@scanproofarg{##3}%
283       \pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
284   \fi
285   \ifdefined\@Opargbegintheorem
286     \let\pgph@orig@Obtp\@Opargbegintheorem
287     \def\@Opargbegintheorem##1##2##3##4{%
288       \pgph@scanproofarg{##2}\pgph@orig@Obtp{##1}{##2}{##3}{##4}}%
289   \fi
290 }

```

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g., `thm:x` for an optional argument [of `Theorem~\ref{thm:x}`], rather than by the most recently opened result. We capture this by wrapping the `proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```

291 \NewDocumentCommand\pgph@zcpproofof{s0{m}}{\proofof{#3}}
292 \newcommand\pgph@scanproofarg[1]{%
293   \begingroup
294     \let\protect\@empty
295     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
296     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
297     \let\zcref\pgph@zcpproofof
298     \setbox\z@\hbox{#1}%
299   \endgroup
300 }

```

To capture the optional title of a detached `amsthm` proof (an optional [of `Theorem~\ref{thm:x}`]) we redefine the outer `\proof` so that, when a title is given, it is scanned and then handed to the *saved* outer macro, which parses it as it always did. Delegating to the saved outer macro rather than to the inner one means we need to know neither the name of the inner macro nor the environment's default title, so the same wrapper serves any proof environment, whatever its default. No recursion occurs: the saved meaning is captured before the redefinition.

We must wrap *only* a genuine optional-argument environment: a package that captures the proof body verbatim (`apxproof`, `myproofs`) redefines `proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When the proof environment is instead a theorem-like environment (Springer

`\spnewtheorem*{proof}`, or a `\newtheorem` of one's own) its title is already handled by `\pgph@hookproofhead`, so there we just install the begin-hook.

```

301 \newcommand\pgph@wrapproofone[1]{%
302   \@ifundefined{#1}{}{%
303     \edef\pgph@proofmeaning{\expandafter\meaning\csname #1\endcsname}%
304     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
305     \IfSubStr\pgph@proofmeaning\pgph@testoptstr
306       {\pgph@redefproof{#1}}%
307       {\AtBeginEnvironment{#1}{\pgph@beginproof}}%
308   }%
309 }
310 \newcommand\pgph@redefproof[1]{%
311   \expandafter\let\csname pgph@savenv@#1\expandafter\endcsname
312     \csname #1\endcsname
313   \expandafter\def\csname #1\endcsname{\pgph@beginproof
314     \@ifnextchar[{\pgph@proofopt{#1}}%
315       {\csname pgph@savenv@#1\endcsname}}%
316 }
317 \def\pgph@proofopt#1[#2]{%
318   \pgph@scanproofarg{#2}\csname pgph@savenv@#1\endcsname[{#2}]}
```

`\proofgraphproofenv`

`\proofgraphproofenv` Which environments are proofs is a list, `proof` to begin with; `\pgph@proofenvs` `\proofgraphproofenv{<names>}` adds to it, for a document whose proofs live in an environment of another name (pf2's structured proofs under a name that avoids the clash with `amsthm`, a `\newtheorem{claimproof}` for the proofs of claims...). A proof is not a result, so the names are excluded from result tracking as well; `\proofgraphtrack` still overrides that for an environment one wants drawn as both.

```

319 \newcommand\pgph@proofenvs{proof}
320 \newcommand\proofgraphproofenv[1]{%
321   \g@addto@macro\pgph@proofenvs{,#1}%
322   \proofgraphtrack{#1}%
323 }
```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. The exemption is for `proof` alone: an environment added with `\proofgraphproofenv` is not one `apxproof` defers, so it is wrapped as usual. This happens at `\begin{document}`, once we know what is loaded and what the preamble has added to the list.

```

324 \newif\ifpgph@apxproof
325 \newcommand\pgph@wrapproofs{%
326   \@for\pgph@one:=\pgph@proofenvs\do{%
327     \edef\pgph@pe{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
328     \ifboolexpr{ bool {pgph@apxproof} and test {\ifdefstring{\pgph@pe}{proof}} }%
329       {}{\expandafter\pgph@wrapproofone\expandafter{\pgph@pe}}%
330   }%
331 }
332 \AtBeginDocument{%
```

```

333 \ifpackageloaded{apxproof}{\pgph@apxprooftrue\pgph@apxinit}{}%
334 \pgph@wrapproofs
335 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `axp@r<n>` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

336 \newcommand\pgph@apxinit{%
337   \def\apxproofhook##1{\pgph@apxproofcapture{##1}}
338 \newcommand\pgph@apxproofcapture[1]{%
339   \edef\pgph@curproof{\pgph@resolve{#1}}%
340   \ifx\pgph@curproof\@empty\else
341     \pgph@installcapture
342   \fi
343 }

```

`\pgph@hookcite` Optional citation capture, applied to `\cite` and, when they are defined (`natbib`, `biblatex`), to `\citet` and `\citep` as well, all sharing one capture chain parameterized by the command name: external references become `cite:-`prefixed target labels, drawn later as distinct nodes. The node identity is the pair (key, note): a `\cite[<note>]{<key>}` inside a proof is distinguished from a `\cite` of the same key with a *different* note, so that citing two different results of the same work (say two theorems of one book) yields two nodes rather than one mislabelled with whichever note was seen first. Each pair is assigned a numeric *slot* (`\pgph@citeslot`); the edge target is `cite:<slot>`, and the slot remembers the key (for the tag and the bibliography hyperlink) and the note (for the label). The note is reduced to a plain string at capture time (fragile wrappers neutralized, `~` normalised to a space), which also serves as the signature so `[Thm 1]` and `[Thm~1]` coincide. We peek for the optional argument with `\@ifnextchar` rather than declaring one, so that a note-less `\cite{<key>}` is passed on *as written*: re-emitting it as `\cite[]{<key>}` would make the typeset citation grow a spurious empty note (`[1, ]`). A second optional argument is peeked for in the same way, to support the two-argument form `\cite[<pre-note>][<note>]{<key>}` of `natbib` and `biblatex`; there the *second* argument is the note recorded for the node (matching the one-argument semantics, where the lone argument is the post-note), while the pre-note (“see”, “e.g.”) qualifies the act of citing rather than identifying the cited result, and is only replayed to the saved `\cite`. Without this peek, the mandatory-argument scan would grab the bare token `[` as the key and typeset the rest of the citation as text, giving confusing errors (“Missing \$ inserted”) deep in the proof body. A star (`\citet*`, `\citep*`: full author list) is likewise peeked for and replayed; it affects rendering only, not the recorded (key, note) pair. As in `\pgph@hooknames`, the notes are replayed braced, `[{<note>}]`, so that a note written `[{p.~37, [sic]}]` to hide a `]` reaches the saved command as written rather than cut short at that `]`. The replay head (saved command plus optional star) is frozen in `\pgph@replay` by the star-peek before the arguments are scanned. As in `\pgph@install`, installation is skipped (per command) when the command is already the capture macro; otherwise a proof nested inside another proof would save the capture as the saved command, and the first citation in the inner proof would recurse forever.

```

344 \newcommand\pgph@gxdefcs[2]{%
345   \global\expandafter\edef\csname#1\endcsname{#2}}
346 \newcommand\pgph@cap@citegen[1]{%
347   \def\pgph@citecmd{#1}%
348   \@ifstar{\pgph@cap@cite@x*}{\pgph@cap@cite@x{}}
349 \def\pgph@cap@cite@x#1{%
350   \edef\pgph@replay{%
351     \expandafter\noexpand\csname pgph@saved@\pgph@citecmd\endcsname#1}%
352   \@ifnextchar[\pgph@cap@cite@opt\pgph@cap@cite@noopt}
353 \def\pgph@cap@cite@opt[#1]{%
354   \@ifnextchar[{\pgph@cap@cite@optii{#1}}{\pgph@cap@cite@opti{#1}}
355 \def\pgph@cap@cite@opti#1#2{%
356   \pgph@cap@citerec{#1}{#2}\pgph@replay[{#1}]{#2}}
357 \def\pgph@cap@cite@optii#1[#2]#3{%
358   \pgph@cap@citerec{#2}{#3}\pgph@replay[{#1}][{#2}]{#3}}
359 \def\pgph@cap@cite@noopt#1{\pgph@cap@citerec{#1}\pgph@replay{#1}}
360 \newcommand\pgph@cap@citerec[2]{%
361   \ifx\pgph@curproof\@empty\else
362     \@for\pgph@one:=#2\do{%
363       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
364       \pgph@citeslot{\pgph@k}{#1}%
365       \pgph@addedge{\pgph@curproof}{cite:\pgph@curslot}}%
366   \fi
367 }
368 \newcommand\pgph@citeslot[2]{%
369   \begingroup
370     \pgph@citesanitize
371     \xdef\pgph@gnotestr{#2}%
372   \endgroup
373   \edef\pgph@sig{#1@@\detokenize\expandafter{\pgph@gnotestr}}%
374   \ifcsname pgph@cslot@\pgph@sig\endcsname
375     \edef\pgph@curslot{\csname pgph@cslot@\pgph@sig\endcsname}%
376   \else
377     \global\advance\pgph@citeslotcnt\@ne
378     \edef\pgph@curslot{\the\pgph@citeslotcnt}%
379     \pgph@gxdefcs{pgph@cslot@\pgph@sig}{\pgph@curslot}%
380     \pgph@gxdefcs{pgph@cskey\pgph@curslot}{#1}%
381     \ifx\pgph@gnotestr\@empty\else
382       \pgph@gxdefcs{pgph@citenote\pgph@curslot}{\pgph@gnotestr}%
383     \fi
384   \fi
385 }

```

`\pgph@citecmds` Every citation command with the signature `\cs*[\langle pre \rangle][\langle post \rangle]{\langle keys \rangle}` is captured by the same `\pgph@cap@citegen`, so which commands are captured is a list rather than code. The default list holds the citation commands of L^AT_EX, natbib and biblatex that cite a work; `\proofgraphcitecommand` adds more. Naming a command that the document never defines costs nothing, as `\pgph@installciteone` installs only over an existing command, so the list may name every spelling a citation package might provide.

The commands printing only a *fragment* of a citation (`\citeauthor`, `\citeyear`...) are in the list as well. They commonly accompany a real citation of the same work, but a repetition is harmless: a (key, note) pair already seen

reuses its slot, and `\pgph@writeedge` draws an edge once, so the second mention of a work adds nothing to the graph. Capturing them means that a proof invoking a work by author or by year alone, which does happen, is not silently left without its edge.

The biblatex *multicite* commands (`\cites`, `\parencites`, `\textcites`) cannot be captured at all: their (*pre*)(*post*) syntax and repeated key groups are a different grammar, which `\pgph@cap@citegen` does not parse.

```

386 \newcommand\pgph@citecmds{%
387   cite,Cite,citet,Citet,citep,Citep,%
388   citealt,Citealt,citealp,Citealp,%
389   parencite,Parencite,textcite,Textcite,autocite,Autocite,%
390   footcite,Footcite,footcitetext,smartcite,Smartcite,supercite,%
391   fullcite,footfullcite,%
392   citeauthor,Citeauthor,citefullauthor,citetitle,Citetitle,%
393   citeyear,Citeyear,citeyearpar,citedate,Citedate,citeurl,citenum}
394 \newcommand\proofgraphcitecommand[1]{\g@addto@macro\pgph@citecmds{,#1}}
395 \newcommand\pgph@installcite{%
396   \@for\pgph@conename:=\pgph@citecmds\do{%
397     \edef\pgph@cone{\expandafter\pgph@trimsp\expandafter{\pgph@conename}}%
398     \expandafter\pgph@installciteone\expandafter{\pgph@cone}}%
399 }

```

The capture macro of a command is built on first use rather than declared: it is the same `\pgph@cap@citegen` call for all of them, and building it only for commands the document actually defines keeps the unused names of the list from costing anything. It must be `\gdef`, the installation happening inside the group of a proof.

```

400 \newcommand\pgph@installciteone[1]{%
401   \ifcsname #1\endcsname
402   \ifcsname pgph@cap@#1\endcsname\else
403     \expandafter\gdef\csname pgph@cap@#1\endcsname{\pgph@cap@citegen{#1}}%
404   \fi
405   \expandafter\ifx\csname #1\endcsname\expandafter\endcsname
406     \csname pgph@cap@#1\endcsname
407   \else
408     \expandafter\let\csname pgph@saved@#1\endcsname\expandafter\endcsname
409     \csname #1\endcsname
410     \expandafter\let\csname #1\endcsname\expandafter\endcsname
411     \csname pgph@cap@#1\endcsname
412   \fi
413 \fi
414 }

```

The biblatex recorder is registered at the end of the preamble, once biblatex has certainly been loaded (it is a preamble-only package) but while `\AtEveryCitekey` still accepts additions. It runs both at every citation and at every bibliography entry, so that a work only `\nocited`, or reached by `\usescite` alone, is still seen. It is not conditioned on the `cite` option, which governs the automatic capture of `\cite` only: `\usescite` and `\proofgraphciteedge` draw citation nodes without it, and those nodes want their tag and their anchor just the same.

```

415 \newif\ifpgph@blx
416 \AtEndPreamble{%
417   \@ifpackageloaded{biblatex}{%

```

```

418 \pgph@blxtrue
419 \AtEveryCitekey{\pgph@blxrecord}%
420 \AtEveryBibitem{\pgph@blxrecord}%
421 }{}%
422 }

```

5.7 User commands

```

423 \newcommand\uses[1]{\pgph@recordref{#1}}

```

`\proofof` names the result a proof establishes. Inside a proof instance it records the label against the instance, so that the pin covers the whole proof, wherever in it the command stands, and is resolved (with every other label) when the graph is written. Outside one – in an `apxproof` deferred proof, whose owning result the `\apxproofhook` already gives, or in the body of a result under `statements` – there is no instance to pin, and the source is redirected on the spot instead.

```

424 \newcommand\proofof[1]{%
425   \ifcsname pgph@powner@\pgph@curproof\endcsname
426     \expandafter\gdef\csname pgph@pofl@\pgph@curproof\endcsname{#1}%
427   \else
428     \ifcsname pgph@map@#1\endcsname
429       \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
430     \fi
431   \fi
432 }
433 \newcommand\proofgraphedge[2]{%
434   \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
435 }
436 \newcommand\pgph@labeledge[2]{%
437   \edef\pgph@ef{\pgph@resolve{#1}}%
438   \ifx\pgph@ef\@empty\else
439     \@for\pgph@one:=#2\do{%
440       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
441       \pgph@plainedge{\pgph@ef}{\pgph@k}}%
442   \fi
443 }

```

`\usescite`

`\proofgraphusesciteedge` The two manual counterparts of the automatic `\cite` capture, for a dependency on external work that no citation in a proof expresses: `\usescite`, the `\uses`
`\proofgraphciteedge` of citation nodes, records one from inside a proof, and `\proofgraphciteedge`,
`\pgph@citelabeledge` the `\proofgraphedge` of citation nodes, states one from anywhere. Both create

the citation node if the work has no node yet, take a comma-separated list of BibTeX keys and accept as an optional argument the note that would have been the optional argument of `\cite` ([Thm~3]), which labels the node and, as in the automatic capture, distinguishes one result of a work from another.

Being explicit, they do not depend on the `cite` option, which only says whether a plain `\cite` is captured on its own. `\usescite` shares the recording path of the captured citation commands, so a work reached both ways still yields a single node.

```

444 \NewDocumentCommand\usescite{0{}m}{\pgph@cap@citerec{#1}{#2}}
445 \NewDocumentCommand\proofgraphciteedge{0{}mm}{%
446   \g@addto@macro\pgph@edges{\pgph@citelabeledge{#1}{#2}{#3}}%

```

447 }

Resolution is deferred to emission, as for `\proofgraphedge`, so the result may be labelled anywhere in the document; the slot is therefore allocated then, which is harmless as slot numbers only have to be distinct. The edge is assembled by `\edef` rather than passed as macros, because `\pgph@citedge` takes the `cite:⟨slot⟩` target apart with `xstring`, which reads its argument literally.

```
448 \newcommand\pgph@citelabeledge[3]{%
449   \edef\pgph@ef{\pgph@resolve{#2}}%
450   \ifx\pgph@ef\@empty\else
451     \@for\pgph@one:=#3\do{%
452       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
453       \pgph@citeslot{\pgph@k}{#1}%
454       \edef\pgph@tmpce{%
455         \noexpand\pgph@citedge{\pgph@ef}{cite:\pgph@curslot}}%
456       \pgph@tmpce}%
457   \fi
458 }
```

```
459 \newcommand\proofgraphignore[2]{%
460   \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
461 }
462 \newcommand\proofgraphexclude[1]{%
463   \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
464 }
465 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}
```

External citation nodes (option `cite`) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```
466 \newcommand\pgph@citestyle{shape=note}
467 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}
```

5.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```
468 \newcommand\pgph@resolve[1]{%
469   \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
470 }
471 \newcommand\pgph@ignorerule[2]{%
472   \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
473   \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
474     \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
475   \fi\fi
476 }
477 \newcommand\pgph@excluderule[1]{%
478   \edef\pgph@x{\pgph@resolve{#1}}%
479   \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
480 }
```

5.9 Emission

`\pgph@dotesc` Escape the two characters that are special in a Graphviz double-quoted string, so a theorem name, number or citation note containing them does not corrupt the `.dot`: a backslash is doubled and a double quote is backslash-escaped (in

that order). `\pgph@bschar` and `\pgph@dqchar` hold the catcode-12 backslash and double quote to search for; `xstring`'s expansion mode is saved and restored around the two substitutions.

```

481 \begingroup
482   \catcode'\|=0 \catcode'\|=12 \catcode'\|=12
483   \gdef\pgph@bschar{\}%
484   \gdef\pgph@dqchar{"}%
485 \endgroup
486 \newcommand\pgph@dotesc[1]{%
487   \saveexpandmode\expandarg
488   \StrSubstitute{#1}{\pgph@bschar}{\pgph@bschar\pgph@bschar}[#1]%
489   \StrSubstitute{#1}{\pgph@dqchar}{\pgph@bschar\pgph@dqchar}[#1]%
490   \restoreexpandmode}

```

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g., a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```

491 \newcommand\pgph@node[3]{%
492   \ifcsname pgph@ex@#1\endcsname\else
493     \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname%
494       \csname pgph@num@#1\endcsname\fi}%
495     \ifx\pgph@nm\empty
496       \ifcsname pgph@force@#2\endcsname
497         \pgph@emitnode{#1}{#2}{#3}%
498       \else
499         \@namedef{pgph@ex@#1}{}%
500       \fi
501     \else
502       \pgph@emitnode{#1}{#2}{#3}%
503     \fi
504   \fi
505 }
506 \newcommand\pgph@emitnode[3]{%
507   \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
508     , \csname pgph@style@#2\endcsname\fi}%
509   \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
510     \csname pgph@name@#1\endcsname\else#3\fi}%
511   \edef\pgph@lbl{\pgph@dn\space\pgph@nm}%
512   \pgph@dotesc\pgph@lbl
513   \immediate\write\pgph@out{%
514     \space\space"n#1" [label="\pgph@lbl"\pgph@s];}%
515   \ifcsname pgph@rmap@#1\endcsname
516     \immediate\write\pgph@mapout{%
517       \string\pgph@nodemap{n#1}{\csname pgph@rmap@#1\endcsname}}%
518   \fi
519 }

```

`\pgph@edge` Emit one edge, resolving both ends and applying the self-loop, ignore and exclude `\pgph@setsrc` filters. Targets beginning with `cite:` denote external nodes.

The source is resolved by `\pgph@setsrc`. An edge captured in a proof has that proof instance as its source, and the result it belongs to is decided here: the label `\proofof` pinned the proof to, when there is one and it resolves, and otherwise the result the proof followed. An unresolvable pin falls back on that result rather than dropping the edge, so a mistyped or never-labelled `\proofof` loses the attribution but not the dependency. An edge recorded outside a proof (the body or the title of a result, an `apxproof` deferred proof) already carries a result id, which is its own source.

```

520 \newcommand\pgph@setsrc[1]{%
521   \edef\pgph@esrc{%
522     \ifcsname pgph@powner@#1\endcsname
523       \csname pgph@powner@#1\endcsname
524     \else#1\fi}%
525   \ifcsname pgph@pofl@#1\endcsname
526     \edef\pgph@epin{\csname pgph@pofl@#1\endcsname}%
527     \edef\pgph@epin{\pgph@resolve{\pgph@epin}}%
528     \ifx\pgph@epin\@empty\else\let\pgph@esrc\pgph@epin\fi
529   \fi
530 }
531 \newcommand\pgph@edge[2]{%
532   \pgph@setsrc{#1}%
533   \ifx\pgph@esrc\@empty\else
534     \expandafter\pgph@edgeone\expandafter{\pgph@esrc}{#2}%
535   \fi
536 }
537 \newcommand\pgph@edgeone[2]{%
538   \IfBeginWith{#2}{cite:}%
539     {\pgph@citeedge{#1}{#2}}%
540     {\pgph@plainedge{#1}{#2}}%
541 }
542 \newcommand\pgph@plainedge[2]{%
543   \edef\pgph@t{\pgph@resolve{#2}}%
544   \ifx\pgph@t\@empty\else
545     \ifcsname pgph@ex@#1\endcsname\else
546       \ifcsname pgph@ex@\pgph@t\endcsname\else
547         \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
548           \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
549             and test {\ifdefstring{\pgph@selfloops}{remove}} }%
550             {\pgph@emitedge{n#1}{n\pgph@t}}%
551           \fi\fi\fi
552   \fi
553 }

```

`\pgph@emitedge` The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads “a is used by b” (i.e. b relies on a); `direction=uses` draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. `\pgph@writeedge` deduplicates.

```

554 \newcommand\pgph@emitedge[2]{%
555   \ifdefstring{\pgph@direction}{uses}%
556     {\pgph@writeedge{#1}{#2}}%
557     {\pgph@writeedge{#2}{#1}}%

```

```

558 }
559 \newcommand\pgph@writeedge[2]{%
560   \ifcsname pgph@drawn@#1@#2\endcsname\else
561     \@namedef{pgph@drawn@#1@#2}{}%
562     \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
563   \fi
564 }

```

`\pgph@citedge` A cite:⟨*slot*⟩ target: recover the slot’s BibT_EX key, declare the external node once per slot, then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the “[42]”/“[Knu74]” text), recovered either from a tag recorded by `\pgph@blxrecord` under biblatex or from the `\b@⟨key⟩` macro that `\bibcite` writes to the .aux (so two runs are needed, and it falls back to the key when no tag is known, e.g., on the first run or with citation packages neither path covers). The slot’s `\cite` note, when present, is added inside the tag. The node-to-label map records the *anchor* of the cited work, so all slots of one work hyperlink to the same bibliography entry.

```

565 \newcommand\pgph@citedge[2]{%
566   \ifcsname pgph@ex@#1\endcsname\else
567     \StrBehind{#2}{cite:}\pgph@slot}%
568   \edef\pgph@key{\csname pgph@cskey@\pgph@slot\endcsname}%
569   \ifcsname pgph@extnode@\pgph@slot\endcsname\else
570     \@namedef{pgph@extnode@\pgph@slot}{}%
571     \expandafter\pgph@setcitelabel\expandafter{\pgph@slot}%
572     \pgph@dotes\pgph@clabel
573     \immediate\write\pgph@out{%
574       \space\space" c@\pgph@slot"
575       [label="\pgph@clabel",\pgph@citestyle];}%
576     \immediate\write\pgph@mapout{%
577       \string\pgph@nodemapcite{c@\pgph@slot}{\pgph@citeanchor{\pgph@key}}}%
578   \fi
579   \pgph@emitedge{#1}{c@\pgph@slot}%
580   \fi
581 }
582 \edef\pgph@obraces{\expandafter\@gobble\string\{ }
583 \newif\ifpgph@natnum
584 \newcommand\pgph@nil{

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@⟨key⟩` yields plain text: `hyperref`’s `\hyper@link` (keep its printed last argument) and the `boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```

585 \newcommand\pgph@citesanitize{%
586   \let\protect\@empty
587   \def\hyper@link[##1]##2##3##4{##4}%
588   \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
589   \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
590   \let\nobreakspace\space \let~\space
591   \def\hbox##1{##1}\def\mbox##1{##1}%
592 }

```

A `natbib` `\b@⟨key⟩` is `{⟨num⟩}{⟨year⟩}{⟨short⟩}{⟨long⟩}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```

593 \def\pgph@natpick#1#2#3#4\pgph@nil{

```

```

594 \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}
595 \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x{}{}\pgph@nil}

```

`\pgph@natnumset` Which of the two `natbib` modes is in force is read off `\ifNAT@numbers`, by comparing it with `\iftrue`. That comparison *must* live in a macro of its own rather than inline in `\pgph@setcitelabel`, because the code around it can be skipped by a false conditional: skipping does not expand anything, it merely counts conditional tokens, and the `\iftrue` here – an operand of `\ifx`, not a conditional – would be counted as one more `\if` to be closed. The skip would then eat one `\fi` too many and run past the end of `\pgph@setcitelabel` into its caller, silently swallowing the very writes that emit the citation node. Inside a macro the token is never seen by the skipping scanner.

```

596 \newcommand\pgph@natnumset{%
597   \pgph@natnumfalse
598   \@ifundefined{ifNAT@numbers}{}{%
599     \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
600     \pgph@natnumtrue\fi}%
601 }

```

`\pgph@blxrecord` `biblatex` writes no `\b@⟨key⟩`: its tags are assembled while the citation is typeset, from data the `.bbl` holds. We therefore record them as they go by, from `\AtEveryCitekey` and `\AtEveryBibitem` (installed below), where the entry is the current one and its fields are readable. `labelnumber` carries the tag of the numeric styles and `labelalpha` that of the alphabetic ones; the author–year styles have neither, as their tag is a formatted name list rather than a field, and are left to the `key` fallback. A field that survives sanitizing with a backslash still in it is not plain text, and is likewise declined.

The same visit records the entry’s *anchor*, the name of the `hyperref` destination its bibliography entry carries, which `biblatex` does not spell `cite.⟨key⟩` as the `LATEX` kernel and `natbib` do but `cite.⟨refsection⟩@⟨key⟩`, the reference section being part of the name because one key may have an entry in each. Naming the destination wrongly is not a visible failure but a misleading one: `hyperref` sends an unknown destination to the end of the document, so the node would link somewhere rather than nowhere. The anchor is recorded whatever `citelabel` says, the hyperlink being drawn either way.

```

602 \newcommand\pgph@blxrecord{%
603   \begingroup
604   \pgph@citesanitize
605   \edef\pgph@blxkey{\thefield{entrykey}}%
606   \pgph@gxdefcs{pgph@blxanchor@pgph@blxkey}%
607     {\the\c@refsection @pgph@blxkey}%
608   \ifdefstring{pgph@citelabel}{tag}{%
609     \edef\pgph@x{\thefield{labelnumber}}%
610     \ifx\pgph@x@empty \edef\pgph@x{\thefield{labelalpha}}\fi
611     \edef\pgph@dx{\detokenize\expandafter{pgph@x}}%
612     \ifx\pgph@x@empty\else
613       \IfSubStr{pgph@dx}{@backslashchar}{%
614         \pgph@gxdefcs{pgph@blxtag@pgph@blxkey}{pgph@x}%
615       }\fi
616     }{}%
617   \endgroup
618 }

```

The anchor of a key never met as a citation nor as a bibliography entry is not known; under `biblatex` we then assume the first reference section, which is the only one most documents have. A key absent from the bibliography has no destination under any name, so nothing is lost when the guess is wrong.

```

619 \newcommand\pgph@citeanchor[1]{%
620   \ifcsname pgph@blxanchor@#1\endcsname
621     \csname pgph@blxanchor@#1\endcsname
622   \else
623     \ifpgph@blx 0@{fi#1%
624   \fi
625 }
```

A note is written the way a citation with a note is printed, inside the tag and after it: [Knu68, pp. 23–27], not the tag after the note. Its dashes are the two ligatures `LATEX` reads as dashes, `--` and `---`, which mean nothing to `Graphviz`; they are turned into the HTML character entities `Graphviz` decodes in an ordinary label, so that a page range set as `[pp.~23--27]` carries the same dash in the graph as in the document. The entities are built with `&` of category 12, as the escapes of `\pgph@dotesc` are, so that nothing in the label depends on the category codes in force where the graph is written.

```

626 \begingroup
627   \catcode'\&=12
628   \gdef\pgph@endash{&ndash;}%
629   \gdef\pgph@emdash{&mdash;}%
630 \endgroup
631 \newcommand\pgph@dashes[1]{%
632   \saveexpandmode\expandarg
633   \StrSubstitute{#1}{---}{\pgph@emdash}[#1]%
634   \StrSubstitute{#1}{--}{\pgph@endash}[#1]%
635   \restoreexpandmode}
636 \newif\ifpgph@ctag
637 \newcommand\pgph@setcitelabel[1]{%
638   \edef\pgph@ckey{\csname pgph@cskey@#1\endcsname}%
639   \edef\pgph@cbase{\pgph@ckey}%
640   \pgph@ctagfalse
641   \ifdefstring{\pgph@citelabel}{tag}{%
642     \ifcsname pgph@blxtag@\pgph@ckey\endcsname
643       \edef\pgph@cbase{\csname pgph@blxtag@\pgph@ckey\endcsname}%
644       \pgph@ctagtrue
645     \else
646       \ifcsname b@\pgph@ckey\endcsname
647         \global\let\pgph@cbaseg\relax
648         \begingroup
649           \pgph@citesanitize
650           \edef\pgph@x{\csname b@\pgph@ckey\endcsname}%
651           \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
652           \IfBeginWith\pgph@dx\pgph@obraceset%
653             \pgph@natnumset
654             \pgph@natdo
655             \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
656           }{}%
657           \ifx\pgph@x\@empty\else
658             \IfSubStr\pgph@dx\@backslashchar{}{}%
```

```

659         \global\let\pgph@cbaseg\pgph@x}%
660     \fi
661 \endgroup
662 \ifx\pgph@cbaseg\relax\else
663     \let\pgph@cbaseg\pgph@cbaseg
664     \pgph@ctagtrue
665 \fi
666 \fi\fi
667 }{}%
668 \edef\pgph@cnote{%
669     \ifcsname pgph@citenote@#1\endcsname
670     , \csname pgph@citenote@#1\endcsname\fi}%
671 \ifx\pgph@cnote\@empty\else\pgph@dashes\pgph@cnote\fi
672 \edef\pgph@clabel{%
673     \ifpgph@ctag[\fi\pgph@cbaseg\pgph@cnote\ifpgph@ctag]\fi}%
674 }

```

`\pgph@render` `autorun` must not re-render a graph that has not changed. Rendering unconditionally is not merely wasteful: the image Graphviz produces is not guaranteed to be reproducible (its `cairo`-based outputs, `pdf` among them, carry a creation date), so an unconditional run makes the embedded image differ at every compilation. A build tool such as `latexmk` then sees an input file that never settles and reruns until it gives up, reporting that too many passes were needed.

We therefore render only when something relevant has changed. The signature of a rendering is the checksum of the `.dot` file just written, together with the engine and the format, which the `.dot` does not record; it is stored in the `.aux` by `\pgph@lastrender` and compared with the signature of the next run. Graphviz is run when the signature differs, when no signature is known yet (first run), when either output file is missing, and, as a safe fallback, when the engine provides no file checksum.

Both signatures are `\detokenized` before being compared. The checksum comes out of `\pdf@filemdfivesum` as characters of category *other*, while the same string read back from the `.aux` has its letters as category *letter*, so an `\ifx` on the raw strings would never see two runs as equal.

```

675 \let\pgph@prevsig\relax
676 \newcommand\pgph@lastrender[1]{\xdef\pgph@prevsig{\detokenize{#1}}}
677 \newcommand\pgph@render{%
678     \immediate\write18{\pgph@engine\space -T\pgph@format\space
679         \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
680     \immediate\write18{\pgph@engine\space -Tplain\space
681         \pgph@file.dot -o \pgph@file-proofgraph.plain}%
682 }
683 \newcommand\pgph@maybrender{%
684     \edef\pgph@sig{\ifdefined\pdf@filemdfivesum
685         \pdf@filemdfivesum{\pgph@file.dot}\fi}%
686     \ifx\pgph@sig\@empty
687         \pgph@render
688     \else
689         \edef\pgph@sig{\pgph@sig-\pgph@engine-\pgph@format}%
690         \edef\pgph@sig{\expandafter\detokenize\expandafter{\pgph@sig}}%
691         \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
692             {\IfFileExists{\pgph@file-proofgraph.plain}%
693              {\ifx\pgph@sig\pgph@prevsig\else\pgph@render\fi}%

```

```

694         {\pgph@render}}}%
695     {\pgph@render}%

```

The signature is recorded whether or not we have just rendered: either way the output files on disk are the ones this `.dot` produces. The write must be `\immediate`, as `\pgph@emit` runs at end of document, where a deferred write would have no shipout left to carry it.

```

696     \if@filesw
697         \immediate\write\@auxout{\string\pgph@lastrender{\pgph@sig}}}%
698     \fi
699 \fi
700 }

```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```

701 \newwrite\pgph@out
702 \newwrite\pgph@mapout
703 \newcommand\pgph@emit{%
704     \immediate\openout\pgph@out=\pgph@file.dot\relax
705     \immediate\openout\pgph@mapout=\pgph@file-proofgraph.map\relax
706     \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
707     \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
708     \begingroup
709         \pgph@ignores
710         \pgph@excludes
711         \pgph@nodes
712         \pgph@edges
713     \endgroup
714     \immediate\write\pgph@out{\pgph@cb}%
715     \immediate\closeout\pgph@out
716     \immediate\closeout\pgph@mapout
717     \ifpgph@autorun\pgph@maybrender\fi
718 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```

719 \AtBeginDocument{\AtEndDocument{\pgph@emit}}

```

If TikZ is available, load its `calc` library now (at `\begin{document}`, where category codes are normal) so that the hyperlink overlay in `\proofgraph` can use it without reading a file mid-document.

```

720 \AtBeginDocument{\ifpackageloaded{tikz}{\usetikzlibrary{calc}}{}}

```

`\pgphnodemap` One entry of the node-to-target map written by `\pgph@emit`: it records, for a `\pgphnodemapcite` Graphviz node identifier, what the node should link to. `\pgphnodemap` records the user *label* of the result a node represents; `\pgphnodemapcite` records the *anchor* of an external citation node, so that (when the cited work is in the same document) the node can link to its bibliography entry, `hyperref`'s `cite.<anchor>` destination, the anchor being the `BIBTEX` key except under `biblatex` (see `\pgph@citeanchor`). The map is read back when the graph is embedded, to turn each node into a hyperlink.

```

721 \newcommand\pgphnodemap[2]{\global\@namedef{pgph@lbl@#1}{#2}}
722 \newcommand\pgphnodemapcite[2]{\global\@namedef{pgph@cbl@#1}{#2}}

```

\proofgraph Include the rendered graph (from a previous run) in `autorun` mode. When `hyperref` and `TikZ` are both available (and `hyperlinks` is not switched off), each node is made into an internal hyperlink to what it represents: a result node links to the result (its `\label`), and an external citation node (from the `cite` option) links to the cited work's bibliography entry, the `cite.<anchor>` destination `hyperref` gave it, when that work is cited in the same document. `Graphviz` produces a `.pdf` whose link annotations would be lost by `\includegraphics` (they sit inside a form `XOBJECT`), so instead we overlay transparent link areas at the node positions, which we read from the `-Tplain` output. Without those packages we fall back to a plain inclusion.

```

723 \newif\ifpgph@canlink
724 \newif\ifpgph@link
725 \newsavebox\pgph@imgbox
726 \newsavebox\pgph@natbox
727 \newread\pgph@plainin
728 \def\pgph@stop{}
729 \newcommand\proofgraph[1][{}]{%
730   \@ifundefined{includegraphics}%
731     {\PackageError{proofgraph}{\string\proofgraph\space requires the
732       graphicx package}{Add \string\usepackage{graphicx} to your preamble
733       to embed the rendered graph.}}%
734     {\IfFileExists{\pgph@file-proofgraph.\pgph@format}%
735       {\pgph@includegraph{#1}}%
736       {\PackageWarning{proofgraph}{Rendered graph
737         \pgph@file-proofgraph.\pgph@format\space not found; compile with
738         shell-escape and the autorun option, then re-run}}}%
739   }
740 \newcommand\pgph@includegraph[1]{%
741   \pgph@canlinkfalse
742   \ifpgph@hyperlinks
743     \@ifpackageloaded{hyperref}%
744       {\@ifpackageloaded{tikz}%
745         {\IfFileExists{\pgph@file-proofgraph.plain}%
746           {\pgph@canlinktrue}{}}}%
747     \fi
748   \ifpgph@canlink
749     \pgph@graphlinked{#1}%
750   \else
751     \includegraphics{#1}{\pgph@file-proofgraph.\pgph@format}%
752   \fi
753 }

```

The linked version. We read the node-to-target map under safe category codes (a label may contain `_` or, under `babel`, an active `:`), measure the rendered image, place it in a `TikZ` node, then read the `-Tplain` file and drop one transparent link box on each mapped node. `Graphviz` draws the graph inset by a margin, so the image is larger than the layout bounding box the `-Tplain` coordinates live in; we recover that margin by also measuring the image at its *natural* size and comparing it with the `-Tplain` graph size (the `graph` line gives it in inches, hence the factor 72). Node positions are then expressed as fractions of the natural image,

so they line up with the drawn nodes whatever size the image is finally scaled to.

```

754 \newcommand\pgph@graphlinked[1]{%
755   \begingroup
756     \catcode'\:=12 \catcode'\_ =12 \catcode'\;=12 \catcode'\!=12
757     \catcode'\?=12 \catcode'\&=12 \catcode'\^=12 \catcode'\~=12
758     \InputIfFileExists{\pgph@file-proofgraph.map}{\fi}%
759   \endgroup
760   \sbox\pgph@imgbox{%
761     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
762   \sbox\pgph@natbox{%
763     \includegraphics{\pgph@file-proofgraph.\pgph@format}}%
764   \edef\pgph@imgw{\the\wd\pgph@imgbox}%
765   \edef\pgph@imght{\the\dimexpr\ht\pgph@imgbox+\dp\pgph@imgbox\relax}%
766   \edef\pgph@natw{\the\wd\pgph@natbox}%
767   \edef\pgph@nath{\the\dimexpr\ht\pgph@natbox+\dp\pgph@natbox\relax}%
768   \begin{tikzpicture}
769     \node[inner sep=\z@,outer sep=\z@](pgph@P){\usebox\pgph@imgbox};
770     \openin\pgph@plainin=\pgph@file-proofgraph.plain\relax
771     \pgph@loopplain
772     \closein\pgph@plainin
773   \end{tikzpicture}%
774 }
775 \newcommand\pgph@loopplain{%
776   \unless\ifeof\pgph@plainin
777     \read\pgph@plainin to \pgph@line
778     \pgph@procline
779     \expandafter\pgph@loopplain
780   \fi
781 }
782 \newcommand\pgph@procline{%
783   \IfBeginWith{\pgph@line}{graph }%
784     {\expandafter\pgph@dograph\pgph@line\pgph@stop}%
785     {\IfBeginWith{\pgph@line}{node }%
786       {\expandafter\pgph@donode\pgph@line\pgph@stop}{}}%
787 }
788 \def\pgph@dograph graph #1 #2 #3 #4\pgph@stop{%
789   \def\pgph@gw{#2}\def\pgph@gh{#3}}
790 \def\pgph@donode node #1 #2 #3 #4 #5 #6\pgph@stop{%
791   \StrDel{#1}{"}[\pgph@nid]%
792   \pgph@linkfalse
793   \ifcsname pgph@lbl@\pgph@nid\endcsname
794     \edef\pgph@tgt{\csname pgph@lbl@\pgph@nid\endcsname}%
795     \edef\pgph@thislink{\noexpand\hyperref[\pgph@tgt]}%
796     \pgph@linktrue
797   \else
798     \ifcsname pgph@clbl@\pgph@nid\endcsname
799       \edef\pgph@tgt{\csname pgph@clbl@\pgph@nid\endcsname}%
800       \edef\pgph@thislink{\noexpand\hyperlink{cite.\pgph@tgt}}%
801       \pgph@linktrue
802   \fi
803   \fi
804   \ifpgph@link
805     % Per-side Graphviz margin: half the natural-minus-layout size.
806     \pgfmathsetmacro\pgph@mx{(\pgph@natw-\pgph@gw*72)/2}%

```

```

807 \pgfmathsetmacro\pgph@my{(\pgph@nath-\pgph@gh*72)/2}%
808 % Node centre as a fraction of the natural image.
809 \pgfmathsetmacro\pgph@fx{(\pgph@mx+#2*72)/\pgph@natw}%
810 \pgfmathsetmacro\pgph@fy{(\pgph@my+#3*72)/\pgph@nath}%
811 \pgfmathsetlengthmacro\pgph@bw{#4*72/\pgph@natw*\pgph@imgw*0.92}%
812 \pgfmathsetlengthmacro\pgph@bh{#5*72/\pgph@nath*\pgph@imgw*0.9}%
813 \node[anchor=center]
814   at ($(\pgph@P.south west)!\pgph@fx!(\pgph@P.south east)$)%
815   !\pgph@fy!($(\pgph@P.north west)!\pgph@fx!(\pgph@P.north east)$)$)
816   {\pgph@thislink{\phantom{\rule{\pgph@bw}{\pgph@bh}}}};%
817 \fi
818 }

819 \end{package}

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols		C	
\!	756	\c@refsection	607
\"	482	\catcode	34, 35, 482, 627, 756, 757
\&	627, 757	\closein	772
\:	756	\Cref	296
\;	756	\cref	295
\?	757	\cs	17
\@M	589	\csundef	93, 101
\@Opargbegintheorem	285, 286, 287	D	
\@auxout	57, 697	\DeclareBoolOption	21, 26, 28, 29
\@backslashchar	613, 658	\DeclareStringOption	20, 22, 23, 24, 25, 27, 30
\@begintheorem	145, 146, 148, 152	\detokenize	304, 373, 611, 651, 655, 676, 690
\@currentlabel	194, 208	\dimexpr	765, 767
\@ifpackageloaded	5, 333, 417, 720, 743, 744	\dp	765, 767
\@ifundefined	6, 10, 302, 598, 730	E	
\@namedef	48, 50, 64, 94, 102, 108, 191, 465, 474, 479, 499, 561, 570, 721, 722	\end	773
\@opargbegintheorem	147, 156, 157, 158, 274, 275, 276	\endproof	7, 8, 11
\@protected@testopt	304	\expandarg	487, 632
\@spbegintheorem	162, 163, 164	\ExplSyntaxOff	18
\@spopargbegintheorem	167, 168, 169, 279, 280, 281	\ExplSyntaxOn	16
\[	34	H	
\]	482	\hbox	179, 298, 591
\{	35, 582	\ht	765, 767
\}	35, 483	\hyper@@link	587
\]	34	\hyperlink	800
\^	757	\hyperref	795
_	756	I	
\	482	\if@filesw	696
\~	757	\IfBeginWith	62, 538, 652, 783, 785
A		\IfBooleanTF	238
\apxproofhook	337	\ifboolexpr	328, 548
\AtBeginDocument	128, 332, 719, 720	\ifdefstring	328, 549, 555, 608, 641
\AtBeginEnvironment	109, 307	\ifeof	776
\AtEndDocument	719	\IfFileExists	691, 692, 734, 745
\AtEndPreamble	416	\ifnumequal	548
\AtEveryBibitem	420	\ifpgph@apxproof	324
\AtEveryCitekey	419	\ifpgph@autorun	717
\autoref	296	\ifpgph@blx	415, 623
\axp@forward@link	187, 207	\ifpgph@canlink	723, 748
B		\ifpgph@cite	259
\begin	768	\ifpgph@ctag	636, 673
		\ifpgph@hyperlinks	742

\ifpgph@link	724, 804	\pgph@canlinkfalse	741
\ifpgph@natnum	583, 594	\pgph@canlinktrue	746
\ifpgph@statements	142	\pgph@cap@autoref	231
\IfSubStr	305, 613, 658	\pgph@cap@cite@noopt	352, 359
\iftrue	599	\pgph@cap@cite@opt	352, 353
\ignorespaces	588	\pgph@cap@cite@opti	354, 355
\includegraphics	751, 761, 763	\pgph@cap@cite@optii	354, 357
\InputIfFileExists	758	\pgph@cap@cite@x	348, 349
J		\pgph@cap@citegen	346, 403
\jobname	32	\pgph@cap@citerec	356, 358, 359, 360, 444
L		\pgph@cap@Cref	230
\label	140, 141	\pgph@cap@cref	229
M		\pgph@cap@eqref	233
\mbox	591	\pgph@cap@ref	228
\meaning	303	\pgph@cap@vref	235
N		\pgph@cap@zceref	236
\newcount	39, 40, 41	\pgph@cb	37, 714
\NewDocumentCommand	236, 291, 444, 445	\pgph@cbase	639, 643, 663, 673
\newif	324, 415, 583, 636, 723, 724	\pgph@cbaseg	647, 659, 662, 663
\newread	727	\pgph@citeanchor	577, 619
\newsavebox	725, 726	\pgph@citecmd	347, 351
\newtheorem	73, 74	\pgph@citecmds	386
\nobreakspace	590	\pgph@citeedge	455, 539, 565
\node	769, 813	\pgph@citelabel	608, 641
\noexpand	137, 217, 351, 455, 795, 800	\pgph@citelabeledge	444
O		\pgph@citesanitize	370, 585, 604, 649
\openin	770	\pgph@citeslot	364, 368, 453
P		\pgph@citeslotcnt	41, 377, 378
\PackageError	731	\pgph@citestyle	466, 467, 575
\PackageWarning	736	\pgph@ckey	638, 639, 642, 643, 646, 650
\pdf@filemdfivesum	684, 685	\pgph@clabel	572, 575, 672
\penalty	589	\pgph@class@endproof	7, 11
\pgfmathsetlengthmacro	811, 812	\pgph@class@proof	7, 11
\pgfmathsetmacro	806, 807, 809, 810	\pgph@cnote	668, 671, 673
\pgph@adddge	216, 225, 365	\pgph@cone	397, 398
\pgph@apxinit	333, 336	\pgph@conename	396, 397
\pgph@apxproofcapture	336	\pgph@ctagfalse	640
\pgph@apxprooftrue	333	\pgph@ctagtrue	644, 664
\pgph@auxmap	48	\pgph@curproof	47, 177, 222, 225, 263, 264, 270, 339, 340, 361, 365, 425, 426, 429
\pgph@beginproof	228, 307, 313	\pgph@curresult	46, 135, 137, 175, 177, 184, 189, 191, 195, 202, 203, 204, 209, 265, 270
\pgph@beginresult	109, 130	\pgph@curslot	365, 375, 378, 379, 380, 382, 455
\pgph@beginstatement	142, 269	\pgph@dashesc	631, 671
\pgph@bh	812, 816	\pgph@defaultenvs	86, 129
\pgph@blxkey	605, 606, 607, 614	\pgph@defaulttexclude	89, 105
\pgph@blxrecord	419, 420, 602	\pgph@direction	555
\pgph@blxtrue	418	\pgph@dn	509, 511
\pgph@bschar	488, 489	\pgph@dograph	784, 788
\pgph@bw	811, 816	\pgph@dohook	106, 114, 116
\pgph@candidates	59, 65, 66, 68, 123, 124		

<code>\pgph@donode</code>	786, 790	<code>\pgph@line</code>	777, 783, 784, 785, 786
<code>\pgph@dotesc</code>	481, 512, 572	<code>\pgph@linkfalse</code>	792
<code>\pgph@dqchar</code>	489	<code>\pgph@linktrue</code>	796, 801
<code>\pgph@dx</code>	611, 613, 651, 652, 655, 658	<code>\pgph@loopplain</code>	771, 775, 779
<code>\pgph@edge</code>	217, 520	<code>\pgph@maplabel</code>	140, 141, 201
<code>\pgph@edgeone</code>	534, 537	<code>\pgph@mapout</code>	516, 576, 702, 705, 716
<code>\pgph@edges</code>	43, 218, 434, 446, 712	<code>\pgph@maybehook</code>	112, 125
<code>\pgph@ef</code>	437, 438, 441, 449, 450, 455	<code>\pgph@maybrender</code>	675, 717
<code>\pgph@emdash</code>	629, 633	<code>\pgph@mx</code>	806, 809
<code>\pgph@emit</code>	701	<code>\pgph@my</code>	807, 810
<code>\pgph@emitedge</code>	550, 554, 579	<code>\pgph@natbox</code>	726, 762, 766, 767
<code>\pgph@emitnode</code>	497, 502, 506	<code>\pgph@natdo</code>	595, 654
<code>\pgph@endash</code>	628, 634	<code>\pgph@nath</code>	767, 807, 810, 812
<code>\pgph@engine</code>	678, 680, 689	<code>\pgph@natnumfalse</code>	597
<code>\pgph@epin</code>	526, 527, 528	<code>\pgph@natnumset</code>	596, 653
<code>\pgph@esrc</code>	521, 528, 533, 534	<code>\pgph@natnumtrue</code>	600
<code>\pgph@excluserule</code>	463, 477	<code>\pgph@natpick</code>	593, 595
<code>\pgph@excludes</code>	45, 463, 710	<code>\pgph@natw</code>	766, 806, 809, 811
<code>\pgph@f</code>	472, 473, 474	<code>\pgph@nid</code>	791, 793, 794, 798, 799
<code>\pgph@file</code>	32, 679, 681, 685, 691, 692, 704, 705, 734, 737, 745, 751, 758, 761, 763, 770	<code>\pgph@nil</code>	584, 593, 595
<code>\pgph@format</code>	678, 679, 689, 691, 734, 737, 751, 761, 763	<code>\pgph@nm</code>	493, 495, 511
<code>\pgph@fx</code>	809, 814, 815	<code>\pgph@node</code>	137, 491
<code>\pgph@fy</code>	810, 815	<code>\pgph@nodes</code>	42, 131, 711
<code>\pgph@gh</code>	789, 807	<code>\pgph@nt@plain</code>	74, 75
<code>\pgph@gnotestr</code>	371, 373, 381, 382	<code>\pgph@nt@star</code>	74, 77
<code>\pgph@graphlinked</code>	749, 754	<code>\pgph@ob</code>	36, 706
<code>\pgph@gw</code>	789, 806	<code>\pgph@obrace</code>	582, 652
<code>\pgph@gxdefcs</code>	344, 379, 380, 382, 606, 614	<code>\pgph@one</code>	91, 92, 93, 94, 95, 99, 100, 101, 102, 120, 121, 122, 124, 125, 223, 224, 326, 327, 362, 363, 439, 440, 451, 452
<code>\pgph@hookcite</code>	344	<code>\pgph@orig@bt</code>	146, 150, 153
<code>\pgph@hooknames</code>	130	<code>\pgph@orig@newtheorem</code>	73, 76, 77
<code>\pgph@hookproofhead</code>	267, 273	<code>\pgph@orig@obt</code>	157, 160
<code>\pgph@idcnt</code>	39, 134, 135	<code>\pgph@orig@Obtp</code>	286, 288
<code>\pgph@ignorerule</code>	460, 471	<code>\pgph@orig@obtp</code>	275, 277
<code>\pgph@ignores</code>	44, 460, 709	<code>\pgph@orig@spbt</code>	163, 165
<code>\pgph@imgbox</code>	725, 760, 764, 765, 769	<code>\pgph@orig@spnewtheorem</code>	79, 83, 84
<code>\pgph@imggh</code>	765, 812	<code>\pgph@orig@spobt</code>	168, 171
<code>\pgph@imgw</code>	764, 811	<code>\pgph@orig@spobtp</code>	280, 283
<code>\pgph@includegraph</code>	735, 740	<code>\pgph@out</code>	513, 562, 573, 701, 704, 706, 707, 714, 715
<code>\pgph@install</code>	240, 253, 254, 255	<code>\pgph@pe</code>	327, 328, 329
<code>\pgph@installcapture</code>	178, 257, 266, 271, 341	<code>\pgph@plainedge</code>	441, 540, 542
<code>\pgph@installcite</code>	259, 386	<code>\pgph@plainin</code>	727, 770, 772, 776, 777
<code>\pgph@installciteone</code>	398, 400	<code>\pgph@prependnode</code>	130, 138
<code>\pgph@installrefs</code>	252, 258	<code>\pgph@prevsig</code>	675, 676, 693
<code>\pgph@k</code>	224, 225, 363, 364, 440, 441, 452, 453	<code>\pgph@procline</code>	778, 782
<code>\pgph@key</code>	568, 577	<code>\pgph@proofcnt</code>	40, 262, 263
<code>\pgph@labeledge</code>	434, 436	<code>\pgph@proofenvs</code>	319, 326
<code>\pgph@lastrender</code>	675	<code>\pgph@proofmeaning</code>	303, 305
<code>\pgph@lbl</code>	511, 512, 514	<code>\pgph@proofopt</code>	314, 317
		<code>\pgph@rankdir</code>	707

<code>\pgph@recordref</code>	221, 228, 229, 230, 232, 234, 235, 237, 423
<code>\pgph@recordresult</code>	130
<code>\pgph@redefproof</code>	306, 310
<code>\pgph@register</code>	59, 76, 83, 95, 122
<code>\pgph@render</code>	675
<code>\pgph@replay</code>	350, 356, 358, 359
<code>\pgph@resolve</code>	339, 437, 449, 468, 472, 478, 527, 543
<code>\pgph@resolverregistration</code> . .	119, 129
<code>\pgph@s</code>	507, 514
<code>\pgph@saved@autoref</code>	232
<code>\pgph@saved@Cref</code>	230
<code>\pgph@saved@cref</code>	229
<code>\pgph@saved@eqref</code>	234
<code>\pgph@saved@ref</code>	228
<code>\pgph@saved@vref</code>	235
<code>\pgph@saved@zcref</code>	238
<code>\pgph@savedlabel</code>	140, 214
<code>\pgph@scanproofarg</code>	277, 282, 288, 291
<code>\pgph@scanrefs</code>	149, 159, 170, 174
<code>\pgph@selfloops</code>	549
<code>\pgph@setcitelabel</code>	565
<code>\pgph@setmap</code>	51, 202
<code>\pgph@setsrc</code>	520
<code>\pgph@sig</code>	373, 374, 375, 379, 684, 686, 689, 690, 693, 697
<code>\pgph@slot</code>	567, 568, 569, 570, 571, 574, 577, 579
<code>\pgph@spnt@plain</code>	81, 82
<code>\pgph@spnt@star</code>	81, 84
<code>\pgph@stop</code>	728, 784, 786, 788, 790
<code>\pgph@t</code>	472, 473, 474, 543, 544, 546, 547, 548, 550
<code>\pgph@testoptstr</code>	304, 305
<code>\pgph@tgt</code>	794, 795, 799, 800
<code>\pgph@thislink</code>	795, 800, 816
<code>\pgph@thisname</code>	61, 62, 63, 64, 66, 68
<code>\pgph@tmp</code>	136, 138
<code>\pgph@tmpce</code>	454, 456
<code>\pgph@tmpedge</code>	217, 219
<code>\pgph@tmphn</code>	192, 193
<code>\pgph@tmpid</code>	52, 53, 54, 55, 57
<code>\pgph@tmpname</code>	188, 190
<code>\pgph@tmpnum</code>	194, 196, 208, 210
<code>\pgph@trimsp</code>	16, 92, 100, 224, 327, 363, 397, 440, 452
<code>\pgph@wrapproof</code>	291
<code>\pgph@wrapproofone</code>	301, 329
<code>\pgph@wrapproofs</code>	325, 334
<code>\pgph@writeedge</code>	554
<code>\pgph@x</code>	478, 479, 594, 595, 609, 610, 611, 612, 614, 650, 651, 655, 657, 659
<code>\pgph@zcproofof</code>	291, 297
<code>\pgphnodemap</code>	517, 721
<code>\pgphnodemapcite</code>	577, 721
<code>\phantom</code>	816
<code>\ProcessKeyvalOptions</code>	31
<code>\proof</code>	7, 8, 11
<code>\proofgraph</code>	8, 723
<code>\proofgraphcitecommand</code>	7, 386
<code>\proofgraphciteedge</code>	6, 25, 444
<code>\proofgraphedge</code>	6, 433
<code>\proofgraphexclude</code>	7, 462
<code>\proofgraphignore</code>	7, 459
<code>\proofgraphproofenv</code>	7, 21, 319
<code>\proofgraphstyle</code>	7, 465
<code>\proofgraphstylecite</code>	8, 467
<code>\proofgraphtrack</code>	7, 13, 86
<code>\proofgraphuntrack</code>	7, 13, 86, 322
<code>\proofof</code>	6, 291, 295, 296, 424
<code>\protect</code>	186, 206, 294, 586
<code>\protected</code>	228, 229, 230, 231, 233, 235
<code>\protected@edef</code>	136, 217
<code>\protected@write</code>	57
R	
<code>\read</code>	777
<code>\ref</code>	295
<code>\restoreexpandmode</code>	490, 635
<code>\rule</code>	816
S	
<code>\saveexpandmode</code>	487, 632
<code>\sbox</code>	760, 762
<code>\setbox</code>	179, 298
<code>\SetupKeyvalOptions</code>	19
<code>\spnewtheorem</code>	78, 79, 80
<code>\StrBehind</code>	567
<code>\StrDel</code>	791
<code>\StrSubstitute</code>	488, 489, 633, 634
T	
<code>\thefield</code>	605, 609, 610
<code>\tl</code>	17
U	
<code>\unexpanded</code>	131
<code>\unhbox</code>	588
<code>\unless</code>	776
<code>\unskip</code>	588
<code>\usebox</code>	769
<code>\usepackage</code>	732
<code>\uses</code>	6, 423
<code>\usescite</code>	6, 25, 444
<code>\usetikzlibrary</code>	720
V	
<code>\voidb@x</code>	589

W	Z
<code>\wd</code>	<code>\z@</code> 179, 298, 769
	<code>\zcref</code> 297

Change History

v0.1.0		
General: Initial version	1	
v1.0.0		
General: First released version . . .	1	
v1.0.1		
General: Capture <code>citet</code> and <code>citep</code> (including starred variants) like <code>cite</code> when they are defined . . .	1	
Fix an infinite loop when, with the <code>cite</code> option, a citation occurred in a proof nested within another proof: the capture no longer reinstalls over itself	1	
Support, with the <code>cite</code> option, the two-optional-argument citation form <code>[pre][post]{key}</code> of <code>natbib</code> and <code>biblatex</code> ; it previously broke compilation with confusing errors inside proofs . .	1	
v1.0.1-dev		
General: Add <code>proofgraphproofenv</code> , declaring further environments to be treated as proofs, for a document whose proofs do not live in an environment named <code>proof</code>	1	
Add <code>usescite</code> and <code>proofgraphciteedge</code> , recording a dependency on cited work by hand, for a proof that does not cite it visibly and for a result stated without a proof to hold a citation	1	
Capture <code>zcref</code> of <code>zref-clever</code> , in its starred, optional-argument and multiple-label forms, like the other cross-referencing commands	1	
Capture citations, and not only cross-references, in the optional title of a result	1	
Capture cross-references and citations in the body of a result and not only in its proof and its optional title, a statement resting on an earlier result being a dependency like any other; the new <code>statements</code> option, on by default, turns this off	1	
Capture, with the <code>cite</code> option, every citation command of <code>natbib</code> and <code>biblatex</code> , such as <code>textcite</code> and <code>parencite</code> , down to those printing a mere fragment of a citation such as <code>citeauthor</code> , and add <code>proofgraphcitecommand</code> to capture further ones	1	
Emit citation nodes again when <code>citelabel=tag</code> finds no tag to use. A stray <code>iftrue</code> , counted as a conditional while a false conditional was being skipped over, made the skip overshoot and swallow the writes emitting the node, which Graphviz then drew from its edges alone: unstyled, labelled by its internal name and without a hyperlink. It affected every first run, and <code>biblatex</code> documents always	1	
Keep the braces of an optional title that is a single brace group, the bracing the <code>amsthm</code> manual prescribes to hide the <code>]</code> of a citation note. A title such as <code>[{cite[p. 37]{a-1}}]</code> reached the heading unbraced, whose own scan for <code>]</code> then stopped inside the note: the citation vanished from the printed title and two errors were reported against the capture	1	
Let <code>proofof</code> pin a proof as a		

whole, wherever in the proof it stands: the references made before it went to whichever result preceded the proof, which a proof deferred to an appendix made easy to hit. The result it names may now be labelled further on in the document	1
Point the hyperlink of a citation node at the destination <code>biblatex</code> gives a bibliography entry, which carries the number of the reference section before the key, rather than at the one the LaTeX kernel and <code>natbib</code> give, which <code>biblatex</code> never defines: the node led to the end of the document instead of the entry	1
Recover the citation tag of the numeric and alphabetic <code>biblatex</code> styles for <code>citelabel=tag</code> , which records no citation tag in the <code>.aux</code> for it to read	1
Require a LaTeX2e format of October 2020 or later, which the use of <code>expl3</code> and <code>NewDocumentCommand</code> already implied	1
Set the note of a citation node inside its tag and after it, “[Knu68, pp. 23–27]”, the way a citation with a note is printed, and render its <code>--</code> and <code>---</code> as the dashes the document shows	1
Trim the spaces after the commas of the lists given to <code>proofgraphtrack</code> and <code>proofgraphuntrack</code> ; every name but the first was silently ignored	1
With <code>autorun</code> , run Graphviz only when the graph, the engine or the format has changed. Rendering at every compilation kept producing a different image, since Graphviz stamps a creation date into its <code>cairo</code> -based outputs, and <code>latexmk</code> would then rerun until it reported that too many passes were needed	1